



**Faculty of Engineering & Technology Electrical
& Computer Engineering Department**

ENCS3320

Computer Networks

Project #1 (Socket Programming)

Prepared by:

NAME : Aya Abdullah	ID: 1220782	SECTION: 1
NAME : Lana Darmna	ID: 1220588	SECTION: 1
NAME : Jeneen Sattof	ID: 1221682	SECTION: 1

Instructor: Alhareth Zyoud

Date : 10/may/2025

Abstract

This project focuses on developing practical skills in socket programming and computer networking through three comprehensive tasks. The first task involves using fundamental network commands and Wireshark to analyze network behavior. The second task is the implementation of a simple web server using Python sockets, serving both English and Arabic web pages with dynamic redirection capabilities. The third task introduces a hybrid TCP/UDP multiplayer guessing game, emphasizing real-time data exchange and synchronization. Each task reinforces core networking concepts and demonstrates the application of theoretical knowledge in a hands-on environment.

Table of Contents

Abstract.....	2
Table of Figures.....	4
List of Tables.....	6
Theory & Procedure	7
Socket Programming	7
A-Socket Programming with UDP.....	7
B-Socket Programming with TCP.....	8
Network Commands Wireshark.....	10
Autonomous Systems (AS) and Tier 1 ISPs.....	11
HTTP Protocol and Web Server.....	11
Results and Discussion.....	13
1) Task 1 – Network Commands and Wireshark.....	13
2) Task 2 – Implement a Simple Web Server Using Socket Programming.....	17
Server in Python code.....	17
web server.....	30
3) Task 3 – TCP/UDP Hybrid Client-Server Game Using Socket Programming.....	44
Server Code	44
Client Code.....	52
Test Cases.....	56
Alternative Solutions, Issues, and Limitations.....	60
Teamwork.....	63
Conclusion.....	64
References.....	65

Table of figures:

Figure 1: User Datagram Protocol.	7
Figure 2:TCP socket programming.	8
Figure 3: UDP socket programming.	9
Figure 4:ipconfig /all command Result (1)	13
Figure 5:ipconfig /all command Result (2)	13
Figure 6 :Smart phone IP	14
Figure 7 :ping Result.	14
Figure 8 :Ping gaia.cs.umass.edu Result.	15
Figure 9 : tracert/traceroute gaia.cs.umass.edu Result.	15
Figure 10 :nslookup gaia.cs.umass.edu.	16
Figure 11 : telnet gaia.cs.umass.edu 80 Result.	16
Figure 12 : Flash cleaning	16
Figure 13: DNS query and response Result(1)	17
Figure 14 :Figure 10: DNS query and response Result(2)	17
Figure 15 : Server Code part one	19
Figure 16 : Server Code part two.	21
Figure 17 :Server Code part three.	22
Figure 18 :Server Code part four.	23
Figure 19 :Server Code part five.	24
Figure 20 :server code part seven	25
Figure 21:server part 8	26
Figure 22 :Server code Part 9.	27
Figure 23 :server code part 10.	28
Figure 24 :server code part 11.	29
Figure 25 :main page in english part 1	30
Figure 26 : :main page in English part 2	30
Figure 27: :main page in english part 3	31
Figure 28 :main page in english part 4	31
Figure 29: :main page in english part 5	32
Figure 30: :main page in english part 6	32
Figure 31 :Run main in English server result	33
Figure 32:main page in arabic part 1	34
Figure 33::main page in Arabic part 2	34
Figure 34 ::main page in Arabic part 3	35
Figure 35 ::main page in Arabic part 4	35
Figure 36::main page in Arabic part 5	36
Figure 37:main page in Arabic part 6	36
Figure 38 :Result of Run Arabic version of main.	37
Figure 39 : MySite_1220588_en.html	37

<i>Figure 40:terminal when run my site</i>	38
<i>Figure 41:Terminal for exist image.</i>	38
<i>Figure 42:image request success</i>	38
<i>Figure 43:image doesn't exist Terminal</i>	39
<i>Figure 44:open google when the image doesn't exist.</i>	39
<i>Figure 45 :Terminal request exist video</i>	39
<i>Figure 46:open the exist video</i>	40
<i>Figure 47:Terminal for search doesn't exist video</i>	40
<i>Figure 48:open YouTube after request doesn't exist video</i>	41
<i>Figure 49:mySite_1220588_ar.html</i>	42
<i>Figure 50:set up UDP&TCP server.</i>	44
<i>Figure 51:Handling UDP&TCP connections</i>	46
<i>Figure 52:Handling TCP clients</i>	47
<i>Figure 53:Handling UDP client</i>	48
<i>Figure 54:function of game</i>	49
<i>Figure 55:remaining of code</i>	50
<i>Figure 56:remaining of code2</i>	51
<i>Figure 57:Initializes the game client</i>	52
<i>Figure 58:Listening from server.</i>	53
<i>Figure 59:Listening from server.</i>	54
<i>Figure 60:Game Loop.</i>	55
<i>Figure 61:test case exit</i>	56
<i>Figure 62:test case Exist (B)</i>	57
<i>Figure 63:test case:Two players join</i>	58
<i>Figure 64:Duplicate names.</i>	58
<i>Figure 65:time up</i>	59
<i>Figure 66:Teamwork</i>	62

List of Table

Table 1:difference between TCP&UDP	8
Table 2:examples of Network Commands	9

Theory & Procedure

Socket Programming

Sockets allow communication of two processes that are running on the same or different machines. Sockets are the end of two-way communication between two programs that are running on the networks.[1]

- Sockets are mostly used in client-server architecture for communication between multiple applications.
- Socket programming tells us how we can use socket API for creating communication between local and remote processes.
- The socket is created by the combination of the IP address and port number of the software. With this combination, the process knows the system address and address of the application where data is to be sent.
- is used to separate IP address and port number. For eg: 192.168.1.67:80, 155.2.12.23:77, etc.

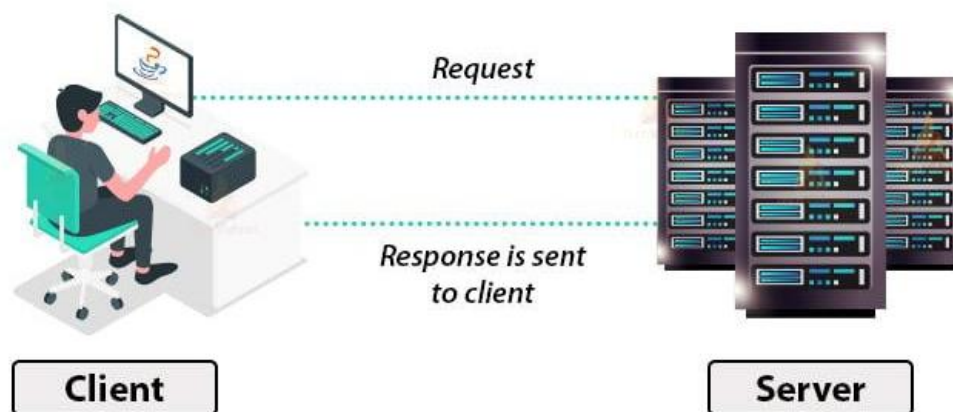


Figure 1: User Datagram Protocol.

A-Socket Programming in TCP

TCP (Transmission Control Protocol) is a reliable, connection-oriented protocol that operates at the transport layer of the OSI model. It ensures the accurate delivery of data between communicating devices by establishing a connection before any data transfer occurs. In TCP socket programming, both the client and server follow specific steps to create, manage, and close connections.

In the Following are steps to be followed for establishing a TCP socket on the **server-side**:

- Use `socket()` for establishing a socket.
- Use the `bind()` function for binding the socket to an address.
- Then for listening client connections use `listen()` function.

- The accept() function is used for accepting the connection of the client.
- Transmit data with the help of the read() and write() function.

In the Following are steps to be followed for establishing a TCP socket on the **client's-side**:

- The first step is to create a socket and use the socket() function to create a socket.
- Use the connect() function for connecting the socket to the server address.
- Transmit data between two communicating parties using read() and write() functions.
- After data transmission completion closes the connection using close() function.

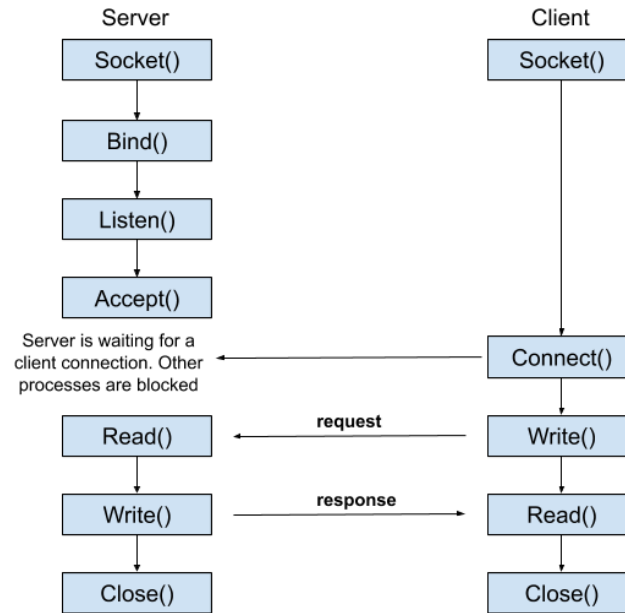


Figure 2:TCP socket programming.

B.Socket Programming in UDP

UDP stands for User Datagram Protocol. UDP is a connection-less and unreliable protocol of transport layer. UDP does not establish a connection between two communicating parties before transmitting the data. Following are the steps given that are to be followed for establishing UDP socket connection on the **client-side**. [1]

- Use socket() function for creating socket;
- recvfrom() and sendto() functions are used for transmitting data between two communicating parties.

Steps to be followed for establishing UDP socket connection at **the server-side**.

- Create a socket using the socket() function.
- Use the bind() function for the binding socket to an address.
- Transmit data with the help of the recvfrom() function and sendto().

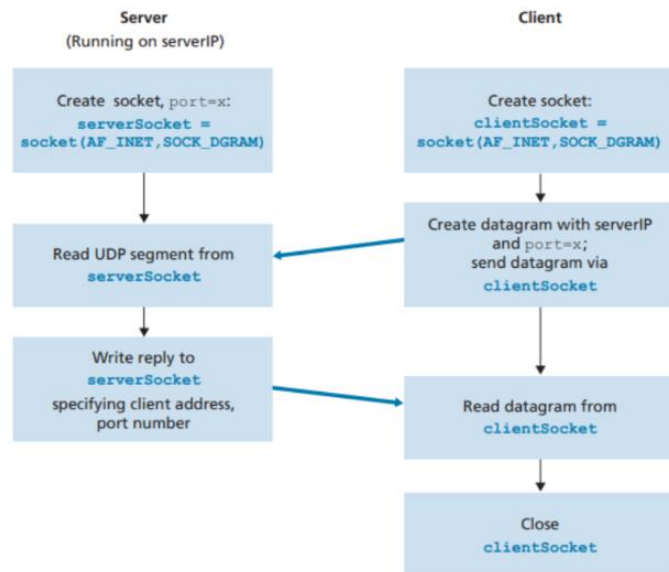


Figure 3: UDP socket programming.

The main difference between TCP (transmission control protocol) and UDP (user datagram protocol) is that TCP is a connection-based protocol and UDP is connectionless. While TCP is more reliable, it transfers data more slowly. UDP is less reliable but works more quickly. This makes each protocol suited to different types of data transfers.[2]

Here are the main differences between TCP and UDP:

Table 1: difference between TCP&UDP

Factor	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable (guaranteed delivery)	Unreliable (no guarantee)
Ordering	Maintains data order	No guaranteed order
Error Handling	Strong error checking and correction	Basic error checking only
Speed	Slower due to overhead	Faster with minimal overhead
Broadcast Support	Not supported	Supported
Use Cases	Web browsing, file transfer, email	Streaming, gaming, VoIP

Network commands & Wireshark

Network commands are essential tools for IT professionals and network engineers. They help diagnose, manage, and troubleshoot complex networking issues efficiently. Even for experienced users, the wide variety of available commands can be overwhelming, and a quick reference is often needed. Below is a summary of some of the most impactful and frequently used network commands that form the foundation of practical networking tasks across different platforms.[3]

Table 2:examples of Network Commands

Command	Purpose in the Project Context
ping	The ping command is used to check if a machine (computer or server) can be accessed via the network. It sends tiny packets of data to the machine and then waits for a response to determine if it's online.
ipconfig (Windows) / ifconfig (Linux)	It shows you your computer's network details, i.e., IP address, subnet mask, and default gateway. It assists you in checking whether your computer is properly connected to the network.
tracert / traceroute	This command illustrates your data traveling to find another device or website. It shows the number of "hops" your data makes through different routers to arrive at your destination.
telnet	Telnet is used to remotely access another device or server on the network, usually for remote management. It is like opening a command-line interface on a different device to run commands remotely.
nslookup	This is a command that enables you to find the IP address of a domain name or website. It comes in useful when you want to know the technical details of a domain, like what server it's hosted on.
netstat	Shows current network connections and ports in use. Useful for verifying active connections during Tasks 2 and 3.
tcpdump (Linux)	Command-line tool for packet capturing (like Wireshark but in terminal). Helpful for observing server-client packet exchange in Task 3.
hostname	Displays the name of the local machine, helpful for identifying devices during testing.
flush DNS (ipconfig /flushdns)	Clears the local DNS cache to ensure fresh DNS queries during testing in Task 1.

Wireshark is a powerful open-source tool used for network protocol analysis and is widely regarded as the industry standard in this field. It is actively maintained by a global community of networking professionals and developers, who continuously update it to support emerging technologies and encryption protocols. Wireshark is used by government agencies, businesses, educational institutions, and non-profits for both troubleshooting and instructional purposes. It offers a practical and detailed way to study network traffic and understand low-level networking operations. However,

it is important to note that Wireshark should only be used on networks where you have explicit permission to capture and analyze traffic, as unauthorized packet inspection is illegal.[4][3]

Wireshark can be used to understand how communication takes place across a network and to analyze what went wrong when an issue in communication arises.

Wireshark helps:

- Network administrators troubleshoot problems across a network
- Security engineers examine security issues across a network
- QA engineers verify applications
- Developers debug protocol implementations
- Network users learn about a specific protocol

Autonomous Systems (AS) and Tier 1 ISPs

Autonomous Systems (AS):

An AS is a collection of IP networks managed by a single organization that presents a common routing policy to the internet. Each AS is assigned a unique number (ASN).

Tier 1 ISPs:

These are top-level Internet Service Providers that can reach every part of the internet without paying for access. They form the backbone of global internet routing and maintain direct connections with other Tier 1 networks.

HTTP Protocol and Web Server

A web server is a combination of hardware and software that uses HTTP and other protocols to respond to client requests on the World Wide Web. Its main role is to store, process, and deliver website content such as text, images, videos, and applications to users' devices. Web servers play a key role in web hosting, allowing websites and web-based applications to be accessible online.

On the hardware side, a web server is a computer connected to the internet, equipped with a processor, memory (RAM), storage drives, and network interface cards. This machine stores all necessary website files, such as HTML documents, JavaScript files, and media, and runs the web server software that manages content delivery. Web server software determines how clients (like web browsers) access and receive this content. It responds to user requests by interpreting domain names and serving the appropriate files through protocols like HTTP. Most web servers also support additional protocols such as SMTP (for email) and FTP (for file transfers).

Overall, the web server acts as a bridge between the user and the hosted website, ensuring fast and secure access to web content across multiple devices including computers, smartphones, and tablets.

How Web Servers Work:[5]

- **Obtain IP Address:** The browser resolves the domain name to an IP address either from its cache or by querying DNS servers.
- **Request the Full URL:** The browser sends a full URL request to the web server.
- **Web Server Response:** The server processes the request and sends back the requested web page or an error message if the page doesn't exist (e.g., 404 Error or 401 Error).
- **Display the Web Page:** The browser receives the content and displays it to the user.

HTTP (Hypertext Transfer Protocol) is the protocol used for communication between web clients (like browsers) and servers.

- **HTTP Request:** Made by the client to request a resource, containing a method (e.g., GET, POST), headers (metadata), and sometimes a body (data).
- **HTTP Response:** Sent by the server with a status code (e.g., 200 OK), headers, and a body (content like HTML or images).
- **Common Status Codes:**
 - **200 OK:** Success.
 - **404 Not Found:** Resource not found.
 - **500 Internal Server Error:** Server issue.
- **HTTP Methods:**
 - **GET:** Retrieve data.
 - **POST:** Send data.
 - **PUT:** Update data.
 - **DELETE:** Remove data.
- **HTTPS:** A secure version of HTTP using encryption.
- **MIME Types:** Define the type of content (e.g., text/html, application/json).

Results & Discussion

Task 1 – Network Commands and Wireshark:

- a) Run the **ipconfig /all** command on your computer and identify the IP address, subnet mask,

default gateway, and Domain Name System (DNS) server addresses for your primary network interface.

```
C:\Users\user>ipconfig /all

Windows IP Configuration

Host Name . . . . . : DESKTOP-2G6AIL2
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No

Ethernet adapter Ethernet:

Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Description . . . . . : Realtek PCIe GbE Family Controller
Physical Address. . . . . : 2C-F0-5D-68-9B-E3
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes

Wireless LAN adapter Local Area Connection* 1:

Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Description . . . . . : Microsoft Wi-Fi Direct Virtual Adapter
Physical Address. . . . . : 34-C9-3D-87-A6-D7
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes

Wireless LAN adapter Local Area Connection* 10:

Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Description . . . . . : Microsoft Wi-Fi Direct Virtual Adapter #2
Physical Address. . . . . : 36-C9-3D-87-A6-D6
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
```

Figure 4:ipconfig /all command Result (1)

```
Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Description . . . . . : Microsoft Wi-Fi Direct Virtual Adapter #2
Physical Address. . . . . : 36-C9-3D-87-A6-D6
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes

Wireless LAN adapter Wi-Fi:

Connection-specific DNS Suffix . :
Description . . . . . : Intel(R) Wi-Fi 6 AX201 160MHz
Physical Address. . . . . : 34-C9-3D-87-A6-D6
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
IPv6 Address. . . . . : fd00::e0a4:b09b:80c:5986(Preferred)
Temporary IPv6 Address. . . . . : fd00::a08d:ddd5:8622:2b5e(Preferred)
Link-local IPv6 Address . . . . . : fe80::831b:7934:38d5:69a6%4(Preferred)
IPv4 Address. . . . . : 192.168.88.10(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Wednesday, May 7, 2025 8:07:29 PM
Lease Expires . . . . . : Wednesday, May 7, 2025 10:35:11 PM
Default Gateway . . . . . : 192.168.88.1
DHCP Server . . . . . : 192.168.88.1
DHCPv6 Iaid . . . . . : 537991087
DHCPv6 Client DUID. . . . . : 00-01-00-01-2D-4C-A3-A8-2C-F0-5D-68-9B-E3
DNS Servers . . . . . : fe80::5ee9:31ff:fe6e:8014%4
                        192.168.88.1
NetBIOS over Tcpip. . . . . : Enabled

Ethernet adapter Bluetooth Network Connection:

Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Description . . . . . : Bluetooth Device (Personal Area Network)
Physical Address. . . . . : 34-C9-3D-87-A6-DA
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
```

Figure 5:ipconfig /all command Result (2)

From the output of the command :

- IP Address (IPv4 Address): 192.168.88.10(Preferred)
 - Subnet Mask: 255.255.255.0
 - Default Gateway: 192.168.88.1
 - DNS Servers:
 - IPv6 Address: fd00::e0a4:b09b:80c:5986(Preferred)
 - (IPv4 Address). : 192.168.88.10(Preferred)
- b) Send a **ping request** to a device within your local network, such as from a laptop to a smartphone connected to the same wireless network.

□ From Next Photo My smart phone IP is (192.168.88.6)



Figure 6 :Smartphone IP

ping 192.168.88.6

```
C:\Users\user>ping 192.168.88.6

Pinging 192.168.88.6 with 32 bytes of data:
Reply from 192.168.88.6: bytes=32 time=62ms TTL=64
Reply from 192.168.88.6: bytes=32 time=67ms TTL=64
Reply from 192.168.88.6: bytes=32 time=77ms TTL=64
Reply from 192.168.88.6: bytes=32 time=90ms TTL=64

Ping statistics for 192.168.88.6:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 62ms, Maximum = 90ms, Average = 74ms
```

Figure 7 :ping Result.

This Result means all packets successfully reached the destination (my smart phone) and were replied to. There was no packet loss.

Ping gaia.cs.umass.edu

```
C:\Users\user>Ping gaia.cs.umass.edu

Pinging gaia.cs.umass.edu [128.119.245.12] with 32 bytes of data:
Reply from 128.119.245.12: bytes=32 time=141ms TTL=40
Reply from 128.119.245.12: bytes=32 time=141ms TTL=40
Reply from 128.119.245.12: bytes=32 time=140ms TTL=40
Request timed out.

Ping statistics for 128.119.245.12:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 140ms, Maximum = 141ms, Average = 140ms
```

Figure 8 :Ping gaia.cs.umass.edu Result.

When I pinged gaia.cs.umass.edu, the test showed that 3 out of 4 packets were successfully returned, with response times around 140-141 ms. This indicates that it took approximately 140 milliseconds for the packets to travel to the server and back. The TTL (Time-to-Live) value was 40, meaning the packets passed through 40 routers before reaching the server. One packet timed out, resulting in 25% packet loss, which suggests a minor network issue or delay. Overall, the responses confirm that the server at gaia.cs.umass.edu is reachable, but there may be some network instability affecting the connection.

tracert/traceroute gaia.cs.umass.edu

```
Command Prompt
C:\Users\user>tracert gaia.cs.umass.edu

Tracing route to gaia.cs.umass.edu [128.119.245.12]
over a maximum of 30 hops:
  0  6 ms  3 ms  3 ms  192.168.88.1
  1  6 ms  7 ms  8 ms  10.74.32.250
  2  55 ms  55 ms  54 ms  10.74.19.29
  3  50 ms  51 ms  49 ms  10.74.19.166
  4  50 ms  49 ms  50 ms  10.74.59.199
  5  58 ms  58 ms  57 ms  ae4-0-13.edge1.Marseille3.Level3.net [213.242.111.121]
  6  57 ms  58 ms  57 ms  ae2-3605.edge4.mrs1.neo.colt.net [171.75.8.225]
  7  *  *  *  Request timed out.
  8  62 ms  61 ms  60 ms  be2779.ccr41.par01.atlas.cogentco.com [154.54.72.109]
  9  67 ms  67 ms  68 ms  be3680.ccr51.lhr01.atlas.cogentco.com [154.54.60.170]
 10  145 ms  146 ms  144 ms  be2393.ccr31.bos01.atlas.cogentco.com [154.54.47.141]
 11  148 ms  145 ms  141 ms  be8038.rcr71.orh02.atlas.cogentco.com [154.54.169.254]
 12  139 ms  138 ms  139 ms  be8628.rcr51.orh01.atlas.cogentco.com [154.54.164.126]
 13  142 ms  142 ms  141 ms  38.104.218.14
 14  143 ms  142 ms  142 ms  69.16.0.8
 15  152 ms  150 ms  144 ms  69.16.1.0
 16  143 ms  141 ms  141 ms  core1-rt-et-8-3-0.gw.umass.edu [192.80.83.109]
 17  165 ms  148 ms  147 ms  n1-rt-1-1-et-0-0-0.gw.umass.edu [128.119.0.216]
 18  142 ms  143 ms  142 ms  n1-fnt-fw-1-1-1-31-vl1692.gw.umass.edu [128.119.77.233]
 19  *  *  *  Request timed out.
 20  152 ms  180 ms  157 ms  core1-rt-et-7-2-1.gw.umass.edu [128.119.0.217]
 21  154 ms  154 ms  173 ms  n5-rt-1-1-xe-2-1-0.gw.umass.edu [128.119.3.33]
 22  149 ms  149 ms  149 ms  c1cs-rt-xe-0-0-0.gw.umass.edu [128.119.3.32]
 23  *  *  *  Request timed out.
 24  142 ms  143 ms  141 ms  gaia.cs.umass.edu [128.119.245.12]

Trace complete.
```

Figure 9 : tracert/traceroute gaia.cs.umass.edu Result.

c) **nslookup gaia.cs.umass.edu**

```
C:\Users\user>nslookup gaia.cs.umass.edu
Server:    UnKnown
Address:   fe80::5ee9:31ff:fe6e:8014

Non-authoritative answer:
Name:      gaia.cs.umass.edu
Address:   128.119.245.12
```

Figure 10 :nslookup gaia.cs.umass.edu.

d) **telnet gaia.cs.umass.edu 80**

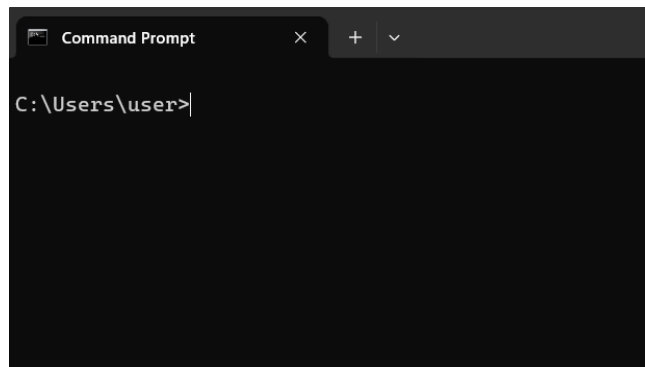


Figure 11 : telnet gaia.cs.umass.edu 80 Result.

We used the telnet command to check if we can talk to the server at gaia.cs.umass.edu on port 80, which is used for websites. The test worked, so this means the server is online and ready to receive web connections.

e) Use **Wireshark** to capture a DNS query and response for gaia.cs.umass.edu (make sure that you clear your cache before requesting this website).

```
C:\Users\user>ipconfig /flushdns

Windows IP Configuration

Successfully flushed the DNS Resolver Cache.
```

Figure 12 : Flush cleaning

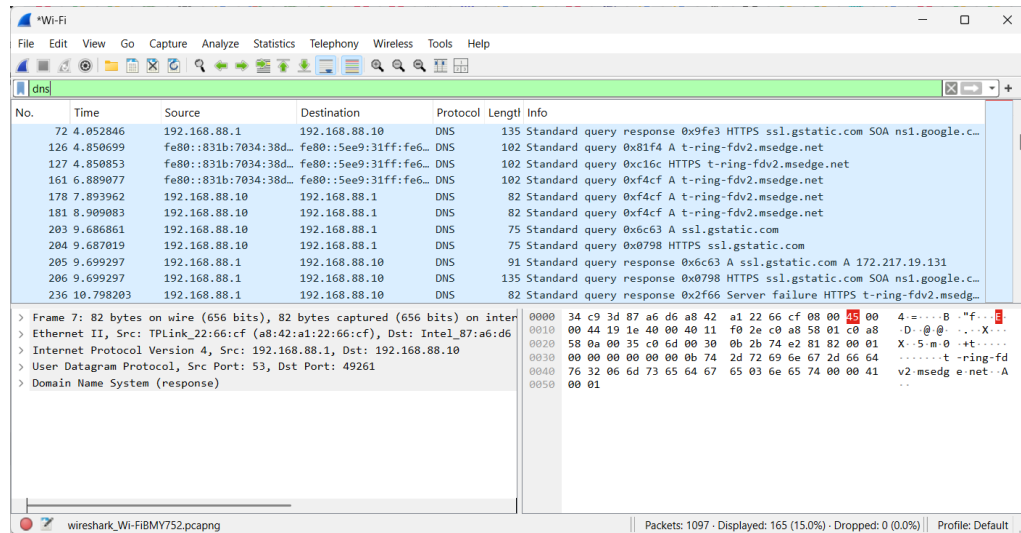


Figure 13: DNS query and response Result(1)

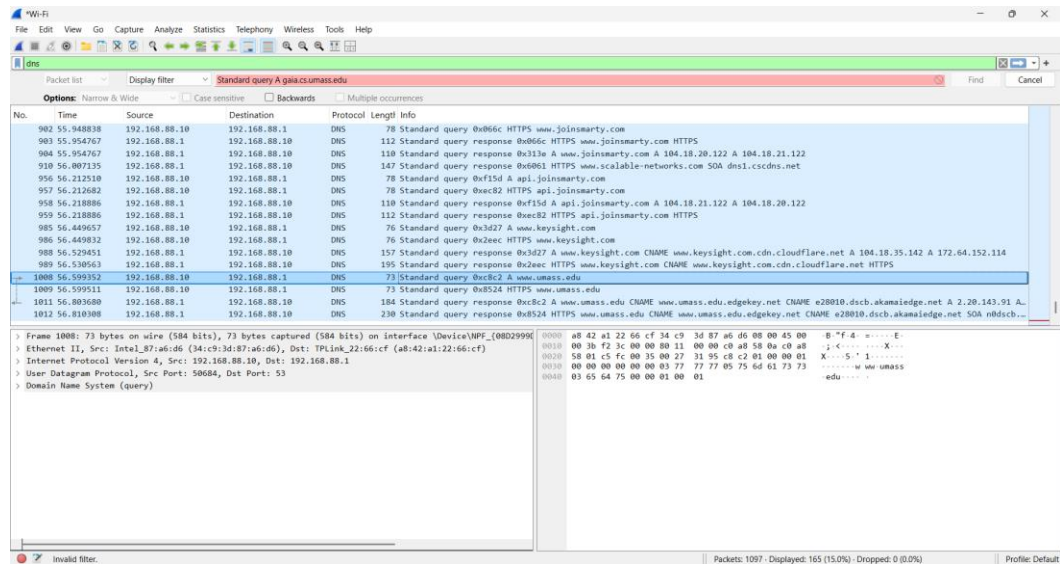


Figure 14 :Figure 10: DNS query and response Result(2)

We used Wireshark to intercept the DNS request and response for the domain gaia.cs.umass.edu. Before we started, we cleared the DNS cache so that the request would go to the DNS server. Upon opening the site after capturing, we saw a "Standard query" for the domain and a "Standard query response" of IP address 128.119.245.12. This confirmed that the DNS request and response were captured correctly.

Task 2 – Implement a Simple Web Server Using Socket Programming:

introduction:

This exercise uses a light-weight HTTP server through Python socket programming on port 9958, serving English and Arabic web pages with CSS styling. The server handles static file serving (HTML, images, videos), form submissions to process media requests, and implements smart fallback strategies - redirecting visitors to Google Images or YouTube in case of missing requested files. Key features include proper HTTP status code handling (200, 307, 404), secure resolution of paths to prevent directory traversal attacks, and enhanced request logging to capture client IP addresses, requested resources, and response codes without sacrificing HTTP/1.1 compliance for good web communication.

Files Overview

main_en.html

- English homepage
- Team profiles (photos, skills, projects)
- Networking lesson: "Network Edge"
- Links: textbook, Birzeit University

main_ar.html

- Arabic homepage
- Same content, RTL layout

mySite_1220588_en.html

- Search form (images/videos)
- Redirect to Google/YouTube if not found

mySite_1220588_ar.html

- Arabic version of search form

server.py

- Processes all HTTP requests
- Serves HTML, image, and video files
- Searches in /images and /videos folders
- Sends 404 and client info on error
- Operates on port 9958 (from student (LANA ID))

1-Server in Python code:

This is a basic HTTP web server in Python using low level socket programming without external libraries like Flask or Django. It hosts static HTML, image, and video content and does basic HTTP requests and responses. It also includes media search fallback via redirects to Google Images and YouTube.

Code and explanations :

```
import os
import socket
from urllib.parse import parse_qs, urlparse
from typing import Tuple, Optional

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
HTML_DIR = os.path.join(BASE_DIR, 'html')
IMAGES_DIR = os.path.join(BASE_DIR, 'images')
VIDEOS_DIR = os.path.join(BASE_DIR, 'videos')
SERVER_PORT = 9958
SERVER_HOST = '0.0.0.0'
MAX_REQUEST_SIZE = 4096
DEFAULT_ERROR_MESSAGE = "<html><body><h1>{status_code} {reason}</h1></body></html>"
```

Figure 15 : Server Code part one

Imports

os: For file system operations like creating directories or retrieving paths.

socket: Core library for building network programs. You use it to build the web server itself.

urllib.parse:

urlparse : divided a URL into parts (e.g., path, query).

parse_qs : Converts the query string (?name=ana&type=image) into a tuple of lists.

typing: Helps to declare function argument types, like "this might return a string or None".

Directory & Port Setup

- **BASE_DIR:** The directory where the script is located.
- **Then it defines folders for:**
 - HTML files (html/)
 - Image files (images/)

- Video files (videos/)
- **The server will run on:**
 - Port 9958 is from “lana id 1220588”
 - All IP addresses of the host machine (0.0.0.0).

3. Server Constants

- Max request size = 4096 bytes (so it doesn’t overload with huge requests).
- HTML if something goes wrong.

```
os.makedirs(HTML_DIR, exist_ok=True)
os.makedirs(IMAGES_DIR, exist_ok=True)
os.makedirs(VIDEOS_DIR, exist_ok=True)

STATUS_REASONS = {
    200: "OK",
    307: "Temporary Redirect",
    400: "Bad Request",
    404: "Not Found",
    500: "Internal Server Error"
}

CONTENT_TYPES = {
    'html': 'text/html',
    'jpg': 'image/jpeg',
    'jpeg': 'image/jpeg',
    'png': 'image/png',
    'gif': 'image/gif',
    'mp4': 'video/mp4',
    'webm': 'video/webm',
    'ico': 'image/x-icon',
    'svg': 'image/svg+xml',
}
```

Figure 16 : Server Code part two.

Create Directories if Not Exist

Creates the html, images, and videos folders if they don’t exist already.

“this added for debug and see if there any error in my dir”

HTTP Status & Content Type Maps

- **STATUS_REASONS :**
Dictionary mapping HTTP status codes to human-readable messages.
- **CONTENT_TYPES :**
Decides which MIME type to serve for each file extension (.html, .jpg, .mp4, etc.).

Safe Path Construction :

```
def get_safe_path(base_dir: str, *path_parts: str):
    """Construct a safe filesystem path within the base directory."""
    try:
        requested_path = os.path.abspath(os.path.join(base_dir, *path_parts))
        base_path = os.path.abspath(base_dir)

        if os.path.commonpath([base_path]) == os.path.commonpath([base_path, requested_path]):
            return requested_path
        return None
    except Exception:
        return None
```

Figure 17 :Server Code part three.

The 'get_safe_path' function serves to construct safely a file system path by concatenating extra path components onto an initial base directory in a manner that the derived path stays under the allowed base directory. It is an essential security test for preventing path traversal attacks, wherein an attacker attempts to access private files outside of the intended directory by manipulating the path (for example, with '/' to move above directories). The approach begins by appending the base directory with the provided sub-paths and subsequently translates the outcome into an absolute path through 'os.path.abspath'. It also translates the base directory into its absolute path. It then checks the common path between the base directory and the requested path. If the common path is the same as the base directory, then the requested path is safely within allowed parameters, and the function returns this safe absolute path. In any other case, or on any error during the process, it returns 'None', effectively denying insecure path requests. This ensures server integrity and data safety.

Get content type

```
def get_content_type(filename: str):  
    """Determine content type based on file extension."""  
    ext = filename.split('.')[-1].lower()  
    return CONTENT_TYPES.get(ext, 'application/octet-stream')
```

Figure 18 :Server Code part four.

The `get_content_type` function is responsible for determining the appropriate MIME (Multipurpose Internet Mail Extensions) type of a file based on its extension. This is essential in web servers, as it informs the browser how to handle and display different types of files (such as HTML pages, images, videos, or stylesheets). The function works by splitting the filename at the period (.) to isolate the file extension, converting it to lowercase to ensure consistency, and then looking it up in the `CONTENT_TYPES` dictionary, which maps known extensions to their correct MIME types. If the extension is recognized, its corresponding content type is returned. If the extension is unknown or not listed in the dictionary, the function returns `'application/octet-stream'` as a fallback, which is the general binary type used for unknown files. This mechanism ensures that the server can correctly label and serve a wide variety of files in a predictable and browser-friendly manner.

Send HTTP Response:

```
def send_response(
    connection: socket.socket, status_code: int, content_type: str = 'text/html',
    content: bytes = b'', headers: Optional[dict] = None
) :
    reason = STATUS_REASONS.get(status_code, "Unknown Status")
    response = f"HTTP/1.1 {status_code} {reason}\r\n"
    response += f"Content-Type: {content_type}\r\n"
    response += f"Content-Length: {len(content)}\r\n"

    if headers:
        for key, value in headers.items():
            response += f"{key}: {value}\r\n"

    response += "\r\n"
    connection.send(response.encode())

    if content:
        connection.send(content)
```

Figure 19 :Server Code part five.

The `send_response` function is responsible for building and sending a complete HTTP response to the client via a socket connection. It takes several parameters: the client socket connection, an HTTP status code (such as 200 for OK or 404 for Not Found), the content type (such as text/html or image/png), the actual content to be sent (as bytes), and any additional headers. The approach begins by retrieving the text reason phrase for the status code (e.g., "OK" or "Not Found") from the `STATUS_REASONS` dictionary. It then constructs the HTTP response header by including the HTTP version, status code, reason phrase, content type, and content length. Other headers provided, if any, are also included in the response. After writing the headers followed by a double line break (`\r\n`), this function sends across this header string across the socket. If any content (i.e., an HTML page's body or the image data), it sends the content as well. This is a function meant to send decent, standards-adhering replies to HTTP queries so that the client (normally a browser) will know what to do with the data obtained.

Send Error Response

```
def send_error(connection: socket.socket, status_code: int):
    try:
        error_message = DEFAULT_ERROR_MESSAGE.format(
            status_code=status_code,
            reason=STATUS_REASONS.get(status_code, "Error")
        ).encode('utf-8')
        send_response(connection, status_code, 'text/html', error_message)
    finally:
        connection.close()
```

Figure 20 :server code part seven

The `send_error` function is employed to handle error conditions by sending the client a proper HTTP error response and then closing the connection. It takes two parameters: the client's socket connection and the HTTP status code for the error type (e.g., 404 for "Not Found" or 500 for "Internal Server Error"). In the process, it constructs a simple HTML error page from a template of the same type that already exists (`DEFAULT_ERROR_MESSAGE`) with the status code and corresponding reason (which is retrieved from the `STATUS_REASONS` dictionary). It bytes-encodes the message (since HTTP responses are also returned as bytes) and invokes the `send_response` method to send it back to the client with the correct content type (`text/html`). Finally, regardless of whether the send succeeds or fails, the `finally` block closes down the socket connection in either event to free resources and to avoid client hanging. This method is a clean and deterministic means of signaling errors to users without affecting server stability.

```
def handle_request(connection: socket.socket, ip: str, port: int, request: str):
    try:
        request_line = request.split('\n')[0].strip()
        print(f"Request: {request_line}")
        parts = request_line.split()
        if len(parts) < 3:
            print(f"Malformed request: {request_line}")
            send_error(connection, 400)
            return
        method, full_path, protocol = parts
        print(f"Request from {ip}:{port} - {method} {full_path}")
        parsed = urlparse(full_path)
        path = parsed.path
        query = parse_qs(parsed.query)
        if path in ["/", "/index.html", "/main_en.html", "/en"]:
            print(f"Serving main_en.html to {ip}:{port}")
            serve_file(connection, 'main_en.html', HTML_DIR)
        elif path in ["/ar", "/main_ar.html"]:
            print(f"Serving main_ar.html to {ip}:{port}")
            serve_file(connection, 'main_ar.html', HTML_DIR)
        elif path == "/mySite_1220588_en.html":
            print(f"Serving mySite_1220588_en.html to {ip}:{port}")
            serve_file(connection, 'mySite_1220588_en.html', HTML_DIR)
        elif path == "/mySite_1220588_ar.html":
            print(f"Serving mySite_1220588_ar.html to {ip}:{port}")
            serve_file(connection, 'mySite_1220588_ar.html', HTML_DIR)
        elif path.startswith("/images/"):
            filename = path.split('/')[-1]
            print(f"Serving image {filename} to {ip}:{port}")
            serve_file(connection, filename, IMAGES_DIR)
        elif path.startswith("/videos/"):
            filename = path.split('/')[-1]
            print(f"Serving video {filename} to {ip}:{port}")
            serve_file(connection, filename, VIDEOS_DIR)
        elif path == "/request_handler":
            print(f"Handling form request from {ip}:{port}")
            handle_form_request(connection, query)
```

Figure 21:server part 8

The `handle_request` function is the server's primary request handler. The function processes incoming HTTP requests by parsing the request line to determine the HTTP method, requested path, and protocol version. If the request is malformed (i.e., does not have the standard three parts), the server responds with a 400 Bad Request. Otherwise, the function uses the requested path to determine which file to serve. Some routes like `/`, `/en`, `/ar`, and their corresponding.html versions are routed to return pre-set HTML pages from the `HTML_DIR`. Similarly, if the route begins with `/images/` or `/videos/`, the function attempts to return the stated media file from the corresponding directory (`IMAGES_DIR` or `VIDEOS_DIR`). If the route is `/request_handler`, it routes handling to a standalone form processing function. For any other URL, the function will try to make an intelligent guess of what content type to use based on the file extension and output out of one of the specified directories. For a non-file or for an unknown extension, it returns 404 Not Found. Any other processing failure returns 500 Internal Server Error. The mechanism also maintains valuable information like the client's IP address and port and served file, so it is not just handy for debugging and monitoring.

Server File:

```
def serve_file(connection: socket.socket, filename: str, base_dir: str):
    try:
        filepath = get_safe_path(base_dir, filename)
        if not filepath or not os.path.exists(filepath):
            print(f"File not found: {filename} in {base_dir}")
            send_error(connection, 404)
            return

        content_type = get_content_type(filename)
        mode = 'rb' if content_type.startswith(('image/', 'video/')) else 'r'

        with open(filepath, mode) as f:
            content = f.read()
            if mode == 'r':
                content = content.encode('utf-8')

        print(f"Sending {filename} (Status: 200) to client")
        send_response(connection, 200, content_type, content)

    except PermissionError:
        print(f"Permission denied for file: {filename}")
        send_error(connection, 403)
    except Exception as e:
        print(f"Error serving file {filename}: {str(e)}")
        send_error(connection, 500)
```

Figure 22 :Server code Part 9.

The `serve_file` function serves to locate and send an requested file to the client. It begins by calling the `get_safe_path` function to return a safe absolute path such that the file to be requested is within the provided `base_dir` to prevent directory traversal attacks. If the file path is invalid or the file does not exist, the server returns a 404 Not Found. It then determines the appropriate content type from the file extension by using the `get_content_type` method. Depending on whether the file is either a binary file (images or videos, for example) or text, the file is read in the appropriate mode (`r` or `rb`) and the content. When one opens a file in text mode, the content is read in as UTF-8 bytes. Then, it opens the file and sends the file to the client with status 200 OK using `send_response`. Two kinds of error cases are dealt with in this function too: if the server lacks permission to read the file, it returns 403 Forbidden error, and if there is any other exception, it returns 500 Internal Server Error. This function secures and correctly sends files using HTTP.

Handle request

```
def handle_form_request(connection: socket.socket, query: dict) :
    try:
        material = query.get('material', [''])[0].strip()
        material_type = query.get('type', [''])[0].strip()
        print(f"Form request - Material: '{material}', Type: '{material_type}'")
        if not material or not material_type:
            print("Missing form parameters")
            send_error(connection, 400)
            return
        if material_type == 'image':
            print(f"Searching for image: {material}")
            found = False
            for ext in ['jpg', 'jpeg', 'png', 'gif']:
                filename = f"{material}.{ext}"
                filepath = get_safe_path(IMAGES_DIR, filename)
                if filepath and os.path.exists(filepath):
                    print(f"Found local image: {filename}")
                    serve_file(connection, filename, IMAGES_DIR)
                    found = True
                    break
            if not found:
                print(f"Redirecting to Google Images for: {material}")
                query_encoded = material.replace(" ", "+")
                redirect_url = f"https://www.google.com/search?q={query_encoded}&tbm=isch"
                headers = {'Location': redirect_url}
                send_response(connection, 307, 'text/html', b'', headers=headers)

        elif material_type == 'video':
            print(f"Searching for video: {material}")
            filename = f"{material}.mp4"
            filepath = get_safe_path(VIDEOS_DIR, filename)
            if filepath and os.path.exists(filepath):
                print(f"Found local video: {filename}")
                serve_file(connection, filename, VIDEOS_DIR)
            else:
                print(f"Redirecting to YouTube for: {material}")
                query_encoded = material.replace(" ", "+")
                redirect_url = f"https://www.youtube.com/results?search_query={query_encoded}"
```

Figure 23 :server code part 10.

The `serve_file` function serves to locate and send an requested file to the client. It begins by calling the `get_safe_path` function to return a safe absolute path such that the file to be requested is within the provided `base_dir` to prevent directory traversal attacks. If the file path is invalid or the file does not exist, the server returns a 404 Not Found. It then determines the appropriate content type from the file extension by using the `get_content_type` method. Depending on whether the file is either a binary file (images or videos, for example) or text, the file is read in the appropriate mode (r or rb) and the content. When one opens a file in text mode, the content is read in as UTF-8 bytes. Then, it opens the file and sends the file to the client with status 200 OK using `send_response`. Two kinds of error cases are dealt with in this function too: if the server lacks permission to read the file, it returns 403 Forbidden error, and if there is any other exception, it returns 500 Internal Server Error. This function secures and correctly sends files using HTTP.

Run Server

```
def run_server() :
    """Run the HTTP server."""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
        server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        server_socket.bind((SERVER_HOST, SERVER_PORT))
        server_socket.listen(5)
        server_socket.settimeout(1)

        print(f"Server started on {SERVER_HOST}:{SERVER_PORT}")
        print(f"HTML directory: {HTML_DIR}")
        print(f"Images directory: {IMAGES_DIR}")
        print(f"Videos directory: {VIDEOS_DIR}")

        try:
            while True:
                try:
                    connection, (ip, port) = server_socket.accept()
                    print(f"New connection from {ip}:{port}")

                    with connection:
                        request = connection.recv(MAX_REQUEST_SIZE).decode()
                        if not request:
                            print("Empty request received")
                            continue

                        handle_request(connection, ip, port, request)

                except socket.timeout:
                    continue
                except Exception as e:
                    print(f"Error handling connection: {str(e)}")

        except KeyboardInterrupt:
            print("\nServer shutting down...")
        finally:
            print("Server stopped")
```

Figure 24 :server code part 11.

The run server function is the function that creates and runs the HTTP server. It uses Python's socket module to create a TCP socket, sets it up for address reuse, binds it to the provided host and port, and begins to listen for incoming connections with a 5 backlog. The socket also is set up to timeout in 1 second to allow the server to periodically check for keyboard interrupts. When started, the server enters an infinite loop to accept and handle client requests. The connection is accepted and client port and IP address are saved. Request data is accepted, decoded, and passed to the handle request function to be processed. The request is removed if null. The server is designed to catch and save any error of connection or processing without causing the application to crash. Upon receiving a Keyboard Interrupt (e.g., Ctrl+C), the server prints a shutdown message and shuts down in an orderly fashion. This is the server's main loop, offering constant and stable operation and client communication management.

2-web server :

1-Main page in English:

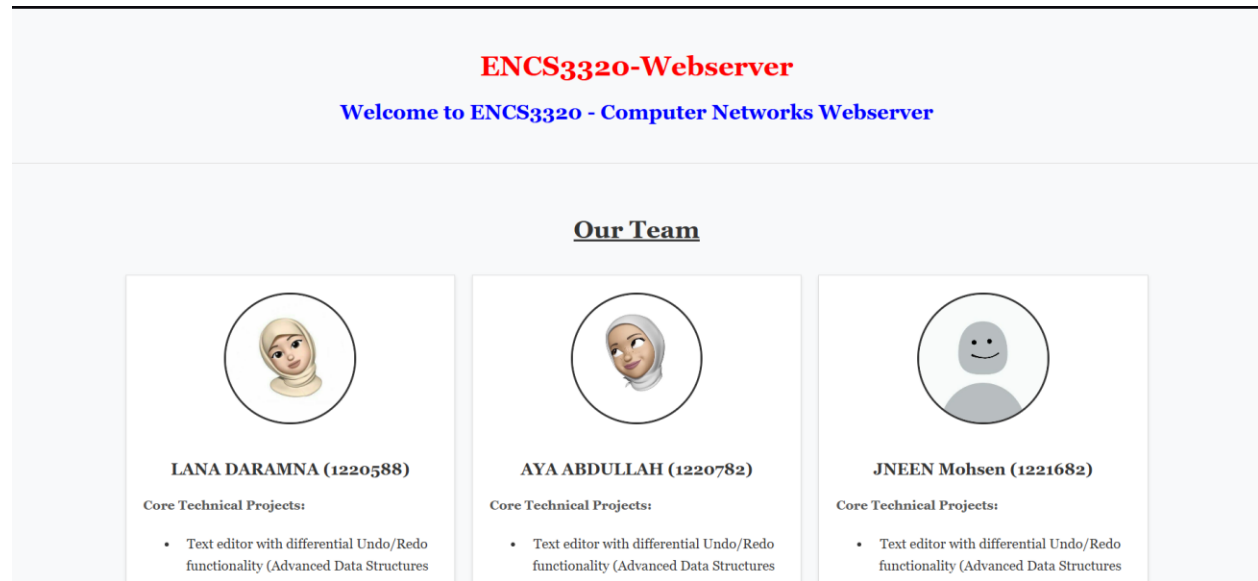


Figure 25 :main page in english part 1

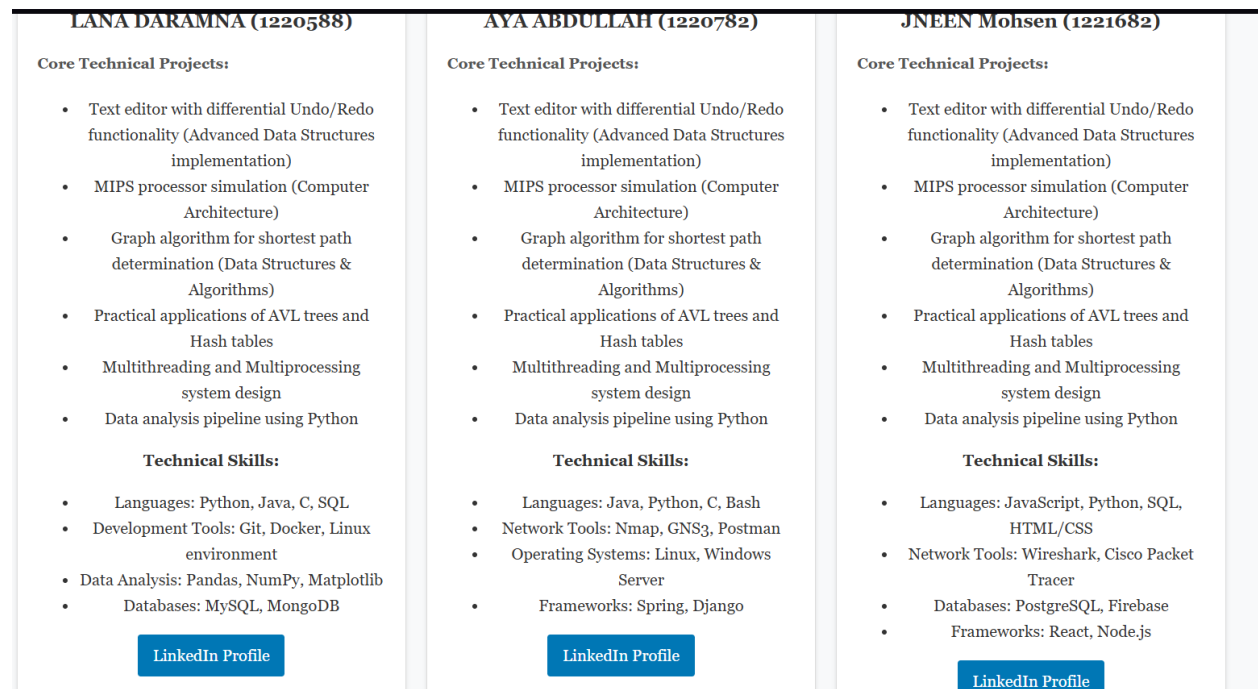
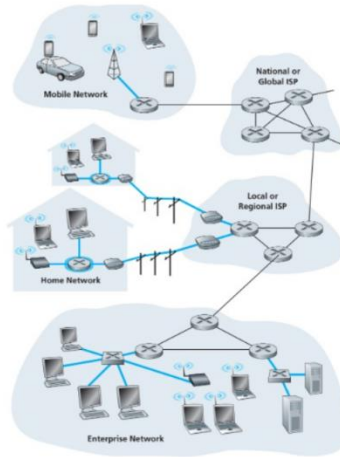


Figure 26 : :main page in English part 2

The Network Edge: End Systems and Access Networks



Key Concept: The network edge consists of end systems (hosts) and the access networks that connect them to the network core.

Figure 27: :main page in english part 3

End Systems (Hosts)

End systems are the devices at the **edge of the Internet** that run application programs. They include:

- Traditional computers (desktops, laptops, servers)
- Mobile devices (smartphones, tablets)
- An increasing array of "Internet of Things" devices:
 - Smart home appliances
 - Wearable technology
 - Industrial sensors
 - Gaming consoles

End systems are also called *hosts* because they host (run) application programs like web browsers and servers. Hosts can be divided into:

1. **Clients:** Typically desktops, laptops, smartphones that request services
2. **Servers:** More powerful machines that provide services (web, email, etc.)

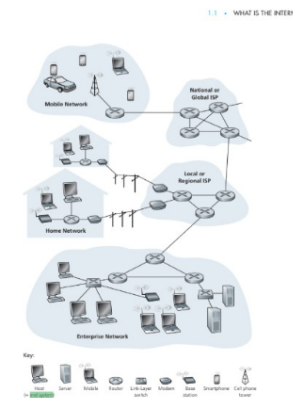


Figure 28 :main page in english part 4

Access Networks

Access networks connect end systems to their **edge routers** (the first router on the path to other networks). The main types include:

<u>Home Access</u> • DSL (Digital Subscriber Line): ◦ Uses existing telephone lines ◦ Asymmetric speeds (faster downstream) ◦ Typical speeds: 12-24 Mbps down, 1.8-2.5 Mbps up • Cable: ◦ Uses cable TV infrastructure ◦ Shared medium (bandwidth varies with neighbors' usage)	<u>Enterprise Access</u> • Ethernet: ◦ Wired connections in offices/campuses ◦ Typical speeds: 100 Mbps to 10 Gbps • WiFi (802.11): ◦ Wireless LAN technology ◦ Typical speeds up to 54 Mbps (higher with newer standards)	<u>Wireless Wide-Area</u> • 3G/4G/LTE: ◦ Cellular network technologies ◦ 3G: 1+ Mbps ◦ 4G/LTE: 10+ Mbps (up to 100s Mbps in ideal conditions) • Satellite: ◦ For remote locations ◦ High latency (280ms for geostationary)
--	---	---

Figure 29: :main page in english part 5

The physical connections that make up access networks and beyond:

Medium	Type	Characteristics	Common Uses
Twisted Pair Copper	Guided	Inexpensive, widely available, limited distance	Telephone lines, Ethernet LANs
Coaxial Cable	Guided	Higher bandwidth than twisted pair, shielding reduces interference	Cable TV, cable Internet
Fiber Optics	Guided	Extremely high bandwidth, low attenuation, immune to EMI	Backbone networks, FTTH
Radio Spectrum	Unguided	No physical connection needed, subject to interference	WiFi, cellular, satellite

Emerging Technologies

- **FTTH (Fiber to the Home):** Provides extremely high speeds (potentially gigabits) directly to residences
- **5G Wireless:** Next generation cellular with speeds rivaling wired connections
- **IoT Networks:** Specialized low-power networks for Internet of Things devices

External Resources

[Textbook Website](#)[Main Arabic Version](#)[Birzeit University](#)[request media english Page](#)[request media arabic Page](#)

© 2025 ENCS3320 Computer Networks - Network Edge Project

Figure 30: :main page in english part 6

Run main in English

After run the server and open main page in English this is the result in terminal:

```
Server started on 0.0.0.0:9958
HTML directory: c:\Users\user\Desktop\Network\project one\html
Images directory: c:\Users\user\Desktop\Network\project one\images
Videos directory: c:\Users\user\Desktop\Network\project one\videos
New connection from 127.0.0.1:59048
Request: GET /main_en.html HTTP/1.1
Request from 127.0.0.1:59048 - GET /main_en.html
Serving main_en.html to 127.0.0.1:59048
Sending main_en.html (Status: 200) to client
New connection from 127.0.0.1:59049
Request: GET /images/LANA.png HTTP/1.1
Request from 127.0.0.1:59049 - GET /images/LANA.png
Serving image LANA.png to 127.0.0.1:59049
Sending LANA.png (Status: 200) to client
New connection from 127.0.0.1:59050
Request: GET /images/AYA.png HTTP/1.1
Request from 127.0.0.1:59050 - GET /images/AYA.png
Serving image AYA.png to 127.0.0.1:59050
Sending AYA.png (Status: 200) to client
New connection from 127.0.0.1:59051
Request: GET /images/JNEEN.png HTTP/1.1
Request from 127.0.0.1:59051 - GET /images/JNEEN.png
Serving image JNEEN.png to 127.0.0.1:59051
Sending JNEEN.png (Status: 200) to client
New connection from 127.0.0.1:59052
Request: GET /images/Network-Edge.jpg HTTP/1.1
Request from 127.0.0.1:59052 - GET /images/Network-Edge.jpg
Serving image Network-Edge.jpg to 127.0.0.1:59052
Sending Network-Edge.jpg (Status: 200) to client
New connection from 127.0.0.1:59053
Request: GET /images/end-system.jpg HTTP/1.1
Request from 127.0.0.1:59053 - GET /images/end-system.jpg
Serving image end-system.jpg to 127.0.0.1:59053
Sending end-system.jpg (Status: 200) to client
```

Figure 31 :Run main in English server result

Explanation:

Once the server had been initiated on 0.0.0.0:9958 successfully, it bootstrapped and displayed the absolute paths of the three major directories: HTML, Images, and Videos. When the main page in English (main_en.html) was accessed from the browser, the server received a new connection from local IP 127.0.0.1:59048. The server interpreted the HTTP GET request, identified the requested file as main_en.html, and responded with a status code of 200, indicating successful transfer of the HTML page.

Subsequently, the browser made other requests automatically to retrieve associated media files that were included in the page. These were files such as LANA.png, AYA.png, JNEEN.png, Network-Edge.jpg, and end-system.jpg. With each of these requests, there was a new connection established, the server noted the request data, looked for the presence of the file within the images folder, and successfully served each of the images with a 200 OK status. This confirms that the server served multiple concurrent requests effectively and reacted accordingly for all static resources.

2-main page in Arabic:

ENCS3320-خادم الويب

مرحبًا بكم في ENCS3320 - خادم ويب لشبكات الحاسوب

فريقنا



جتنين محسن (1221682)

المشاريع التقنية الأساسية:

- محرر نصوص مع وظيفة التراجع/الإعادة التفاضلية (تنفيذ هيكل البيانات المتقدمة)
- محاكاة معالج MIPS (هندسة الحاسوب)
- خوارزمية الرسم البياني لتحديد أقصر مسار (هيكل البيانات والخوارزميات)



آية عبد الله (1220782)

المشاريع التقنية الأساسية:

- محرر نصوص مع وظيفة التراجع/الإعادة التفاضلية (تنفيذ هيكل البيانات المتقدمة)
- محاكاة معالج MIPS (هندسة الحاسوب)
- خوارزمية الرسم البياني لتحديد أقصر مسار (هيكل البيانات والخوارزميات)



لاتا درامنة (1220588)

المشاريع التقنية الأساسية:

- محرر نصوص مع وظيفة التراجع/الإعادة التفاضلية (تنفيذ هيكل البيانات المتقدمة)
- محاكاة معالج MIPS (هندسة الحاسوب)
- خوارزمية الرسم البياني لتحديد أقصر مسار (هيكل البيانات والخوارزميات)

Figure 32:main page in arabic part 1

المهارات التقنية:

- اللغات: JavaScript, Python, SQL, HTML/CSS
- أدوات الشبكة: Wireshark, Cisco Packet Tracer
- قواعد البيانات: PostgreSQL, Firebase
- أطر العمل: React, Node.js

الملف الشخصي على LinkedIn

المهارات التقنية:

- اللغات: Java, Python, C, Bash
- أدوات الشبكة: Nmap, GNS3, Postman
- أنظمة التشغيل: Linux, Windows Server
- أطر العمل: Spring, Django

الملف الشخصي على LinkedIn

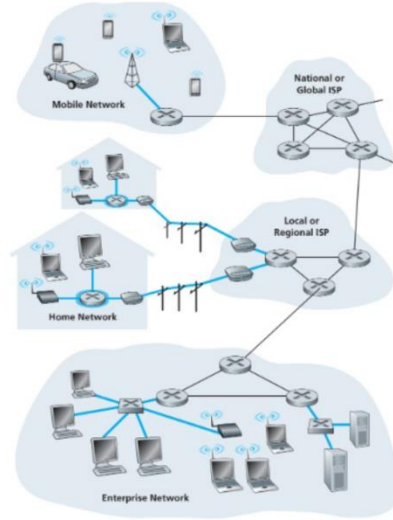
المهارات التقنية:

- اللغات: Python, Java, C++, SQL
- أدوات التطوير: Git, Docker, بيئة Linux
- تحليل البيانات: Pandas, NumPy, Matplotlib
- قواعد البيانات: MySQL, MongoDB

الملف الشخصي على LinkedIn

Figure 33::main page in Arabic part 2

32



المفهوم الأساسي: تتكون حافة الشبكة من أنظمة النهاية (المضيفات) وشبكات الوصول التي تربطها بنواة الشبكة.

Figure 34 ::main page in Arabic part 3

أنظمة النهاية (المضيفات)

أنظمة النهاية هي الأجهزة الموجودة في حافة الإنترنت التي تقوم بتشغيل برامج التطبيقات. تشمل:

- أجهزة الحاسوب التقليدية (أجهزة سطح المكتب، أجهزة الحاسوب المحمولة، الخوادم)
- الأجهزة المحمولة (الهواتف الذكية، الأجهزة اللوحية)
- مجموعة متزايدة من أجهزة "إنترنت الأشياء":
 - الأجهزة المنزلية الذكية
 - التقنيات القابلة للارتداء
 - أجهزة الاستشعار الصناعية
 - أجهزة الألعاب

تسمى أنظمة النهاية أيضًا **المضيفات** لأنها تستضيف (تشغل) برامج التطبيقات مثل متصفحات الويب والخوادم. يمكن تقسيم المضيفات إلى:

1. **العملاء:** عادة ما تكون أجهزة سطح المكتب، أجهزة الحاسوب المحمولة، الهواتف الذكية التي تطلب الخدمات
2. **الخوادم:** أجهزة أكثر قوة توفر الخدمات (الويب، البريد الإلكتروني، إلخ)

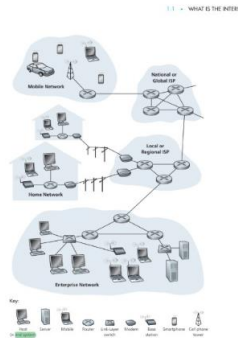


Figure 35 ::main page in Arabic part 4

شبكات الوصول

تربط شبكات الوصول أنظمة النهاية بموجهات الحافة الخاصة بها (أول موجه في المسار إلى الشبكات الأخرى). تشمل الأنواع الرئيسية:

الوصول المنزلي	الوصول المؤسسي	شبكات واسعة النطاق لاسلكية
• DSL (خط المشترك الرقمي): ○ يستخدم خطوط الهاتف الحالية ○ سرعات غير متماثلة (أسرع في اتجاه التنزيل) ○ السرعات النموذجية: 12-24 ميغابت في الثانية ○ للتنزيل، 1.8-2.5 ميغابت في الثانية للرفع • الكابل: ○ يستخدم بنية تحتية لتلفزيون الكابل ○ وسط مشترك (يختلف النطاق الترددي مع استخدام الجيران)	• Ethernet: ○ اتصالات سلكية في المكاتب/الحرم الجامعي ○ السرعات النموذجية: 100 ميغابت في الثانية إلى 10 جيجابت في الثانية • WiFi (802.11): ○ تقنية الشبكة المحلية اللاسلكية ○ سرعات نموذجية تصل إلى 54 ميغابت في الثانية (أعلى مع المعايير الأحدث)	• 3G/4G/LTE: ○ تقنيات الشبكات الخلوية ○ 1: 3G+ ميغابت في الثانية ○ 10: 4G/LTE+ ميغابت في الثانية (حتى 100 ميغابت في الثانية في الظروف المثالية) • الأقمار الصناعية: ○ للمواقع النائية ○ كتم عالي (280 مللي ثانية تأخير للأقمار الصناعية الثابتة بالنسبة للأرض)

Figure 36::main page in Arabic part 5

الوصلات المادية التي تشكل شبكات الوصول وما بعدها:

الوسط	النوع	الخصائص	الاستخدامات الشائعة
زوج النحاس الملتوي	موجه	غير مكلف، متاح على نطاق واسع، مسافة محدودة	خطوط الهاتف، شبكات Ethernet المحلية
الكابل المحوري	موجه	نطاق ترددي أعلى من الزوج الملتوي، التدرج يقلل من التداخل	تلفزيون الكابل، الإنترنت عبر الكابل
الألياف الضوئية	موجه	نطاق ترددي عالي جدًا، توهين منخفض، مقاوم للتداخل الكهرومغناطيسي	شبكات العمود الفقري، FTTH
طيف الراديو	غير موجه	لا حاجة إلى اتصال مادي، عرضة للتداخل	WiFi، الشبكات الخلوية، الأقمار الصناعية

التقنيات الناشئة

- **FTTH (الألياف الضوئية إلى المنزل):** يوفر سرعات عالية جدًا (يحتمل أن تكون جيجابت) مباشرة إلى المنازل
- **الشبكات اللاسلكية 5G:** الجيل التالي من الشبكات الخلوية بسرعات تتنافس الاتصالات السلكية
- **شبكات إنترنت الأشياء:** شبكات متخصصة منخفضة الطاقة لأجهزة إنترنت الأشياء

موارد خارجية

طلب الوسائط بالإنجليزية

طلب الوسائط بالعربية

الصفحة الرئيسية باللغة الإنجليزية

جامعة بيرزيت

الكتاب

© 2025 ENCS3320 شبكات الحاسوب - مشروع حافة الشبكة

Figure 37:main page in Arabic part 6

Run main in Arabic from English page:

```
[2025-05-11 15:47:53] INFO: Request: GET /images/end-system.jpg from 127.0.0.1:59101
[2025-05-11 15:47:53] INFO: Served file: end-system.jpg
[2025-05-11 15:47:59] DEBUG: New connection from 127.0.0.1:59103
[2025-05-11 15:47:59] INFO: Request: GET /main_ar.html from 127.0.0.1:59103
[2025-05-11 15:47:59] INFO: Served file: main_ar.html
[2025-05-11 15:47:59] DEBUG: New connection from 127.0.0.1:59104
[2025-05-11 15:47:59] INFO: Request: GET /images/LANA.png from 127.0.0.1:59104
[2025-05-11 15:47:59] INFO: Served file: LANA.png
[2025-05-11 15:47:59] DEBUG: New connection from 127.0.0.1:59105
[2025-05-11 15:47:59] INFO: Request: GET /images/AYA.png from 127.0.0.1:59105
[2025-05-11 15:47:59] INFO: Served file: AYA.png
[2025-05-11 15:47:59] DEBUG: New connection from 127.0.0.1:59106
[2025-05-11 15:47:59] INFO: Request: GET /images/JNEEN.png from 127.0.0.1:59106
[2025-05-11 15:47:59] INFO: Served file: JNEEN.png
[2025-05-11 15:47:59] DEBUG: New connection from 127.0.0.1:59107
[2025-05-11 15:47:59] INFO: Request: GET /images/Network-Edge.jpg from 127.0.0.1:59107
[2025-05-11 15:47:59] INFO: Served file: Network-Edge.jpg
[2025-05-11 15:47:59] DEBUG: New connection from 127.0.0.1:59108
[2025-05-11 15:47:59] INFO: Request: GET /images/end-system.jpg from 127.0.0.1:59108
[2025-05-11 15:47:59] INFO: Served file: end-system.jpg
```

Figure 38 :Result of Run Arabic version of main.

3-MySite English version:

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:9958/mysite_1220588_en.html'. The main content area has a title 'Computer Networking Media Request'. Below the title is a form with two input fields: 'File Name:' with a placeholder 'e.g., Network-Edge' and 'Media Type:' with a dropdown menu showing '-- Select Media Type --'. A blue 'Request Media' button is positioned below these fields. At the bottom of the form are three buttons: 'Back to Main Page', 'Back to Main Page "arabic type"', and 'request media "arabic"'. Below the form is a light blue box titled 'How to use this form:' containing instructions: 'Enter the name of the computer networking image or video you're looking for and select the media type. If the file is not available on our server, you'll be redirected to Google search results for your request.' followed by a bulleted list: '• For images: You'll be redirected to Google Images' and '• For videos: You'll be redirected to YouTube'.

Figure 39 : MySite_1220588_en.html

Run MySite_1220588_en.html :

```
[2025-05-11 16:01:57] DEBUG: New connection from 127.0.0.1:59194
[2025-05-11 16:01:57] INFO: Request: GET /mySite_1220588_en.html from 127.0.0.1:59194
[2025-05-11 16:01:58] INFO: Served file: mySite_1220588_en.html
[2025-05-11 16:01:58] DEBUG: New connection from 127.0.0.1:59195
[2025-05-11 16:03:43] DEBUG: Empty request received
```

Figure 40:terminal when run my site

Test of this code :

1- Test of exist image

```
[2025-05-11 15:47:59] INFO: Request: GET /images/end-system.jpg from 127.0.0.1:59108
[2025-05-11 15:47:59] INFO: Served file: end-system.jpg
[2025-05-11 16:01:57] DEBUG: New connection from 127.0.0.1:59194
[2025-05-11 16:01:57] INFO: Request: GET /mySite_1220588_en.html from 127.0.0.1:59194
[2025-05-11 16:01:58] INFO: Served file: mySite_1220588_en.html
[2025-05-11 16:01:58] DEBUG: New connection from 127.0.0.1:59195
[2025-05-11 16:03:43] DEBUG: Empty request received
[2025-05-11 16:06:21] DEBUG: New connection from 127.0.0.1:59255
[2025-05-11 16:06:21] INFO: Request: GET /request_handler?material=end-system&type=image from 127.0.0.1:59255
[2025-05-11 16:06:21] INFO: Form request - material: end-system, type: image
[2025-05-11 16:06:21] INFO: Served file: end-system.jpg
[2025-05-11 16:06:21] DEBUG: New connection from 127.0.0.1:59256
```

Figure 41:Terminal for exist image.

And its open the image

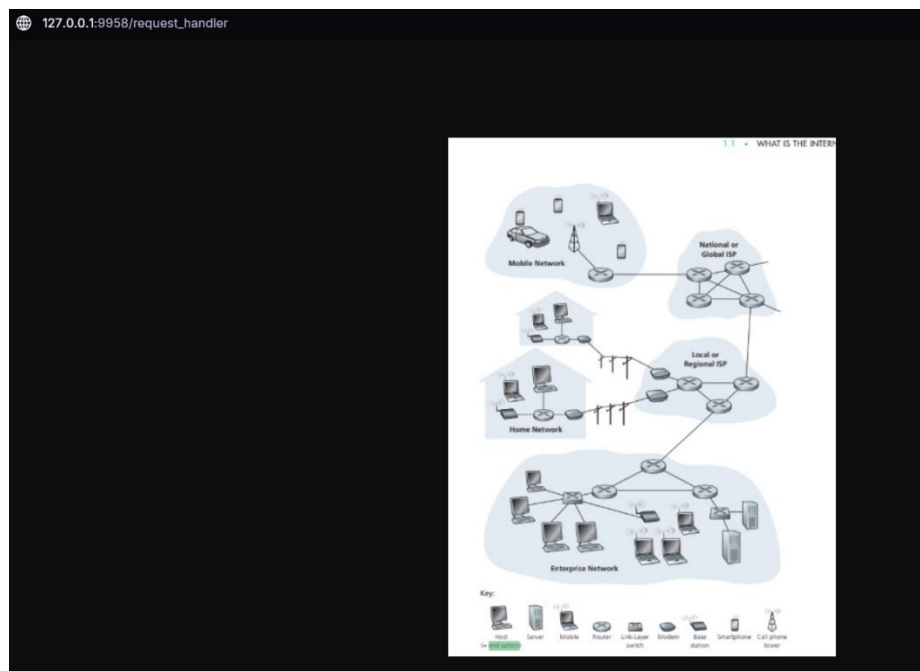


Figure 42:image request success

2- Test of image not exist :

```

[2025-05-11 16:06:21] DEBUG: New connection from 127.0.0.1:59256
[2025-05-11 16:09:17] DEBUG: Empty request received
[2025-05-11 16:09:17] DEBUG: New connection from 127.0.0.1:59260
[2025-05-11 16:09:29] INFO: Request: GET /request_handler?material=network&type=image from 127.0.0.1:59260
[2025-05-11 16:09:29] INFO: Form request - material: network, type: image
[2025-05-11 16:09:29] INFO: Redirecting to Google Images for: network
[2025-05-11 16:09:29] DEBUG: New connection from 127.0.0.1:59261

```

Figure 43: image doesn't exist Terminal

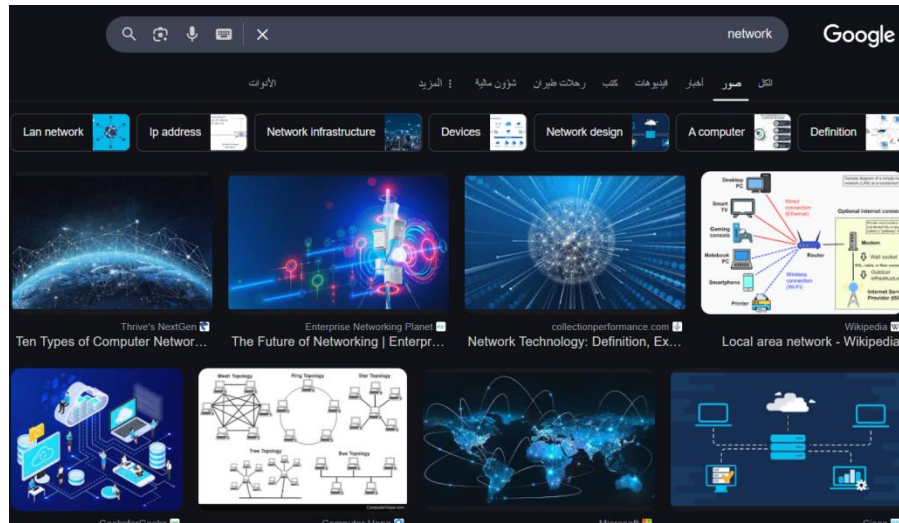


Figure 44: open google when the image doesn't exist.

3- Search for video is exist.

```

[2025-05-11 16:19:24] INFO: Form request - material: vedio_one, type: video
[2025-05-11 16:19:24] INFO: Served file: vedio_one.mp4
[2025-05-11 16:19:24] DEBUG: New connection from 127.0.0.1:59361
[2025-05-11 16:19:24] INFO: Request: GET /request_handler?material=vedio_one&type=video from 127.0.0.1:59361
[2025-05-11 16:19:24] INFO: Form request - material: vedio_one, type: video
[2025-05-11 16:19:24] INFO: Served file: vedio_one.mp4
[2025-05-11 16:19:35] DEBUG: New connection from 127.0.0.1:59362

```

Figure 45 :Terminal request exist video

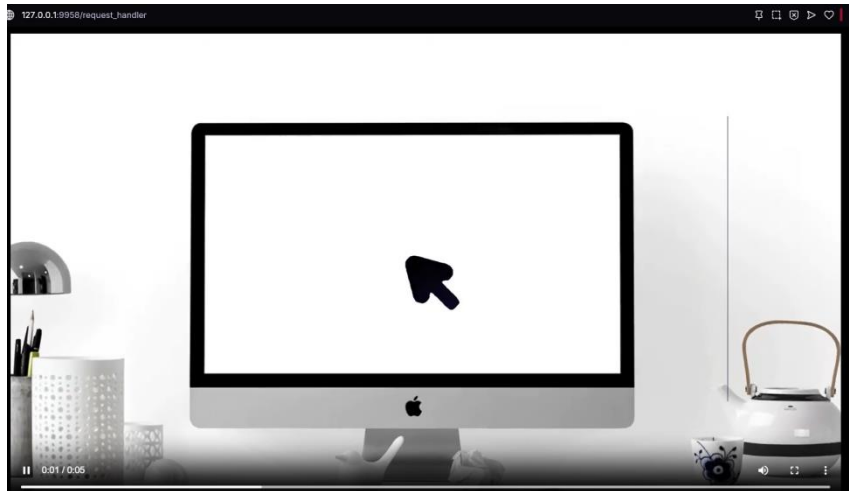


Figure 46: open the exist video

4- Request doesn't exist video:

```
[2025-05-11 16:19:24] INFO: Served File: video_one.mp4
[2025-05-11 16:19:35] DEBUG: New connection from 127.0.0.1:59362
[2025-05-11 16:20:15] DEBUG: Empty request received
[2025-05-11 16:22:40] DEBUG: New connection from 127.0.0.1:59371
[2025-05-11 16:22:40] INFO: Request: GET /request_handler?material=network&type=video from 127.0.0.1:59371
[2025-05-11 16:22:40] INFO: Form request - material: network, type: video
[2025-05-11 16:22:40] INFO: Redirecting to YouTube for: network
[2025-05-11 16:22:40] DEBUG: New connection from 127.0.0.1:59372
[2025-05-11 16:22:45] DEBUG: Empty request received
```

Figure 47: Terminal for search doesn't exist video

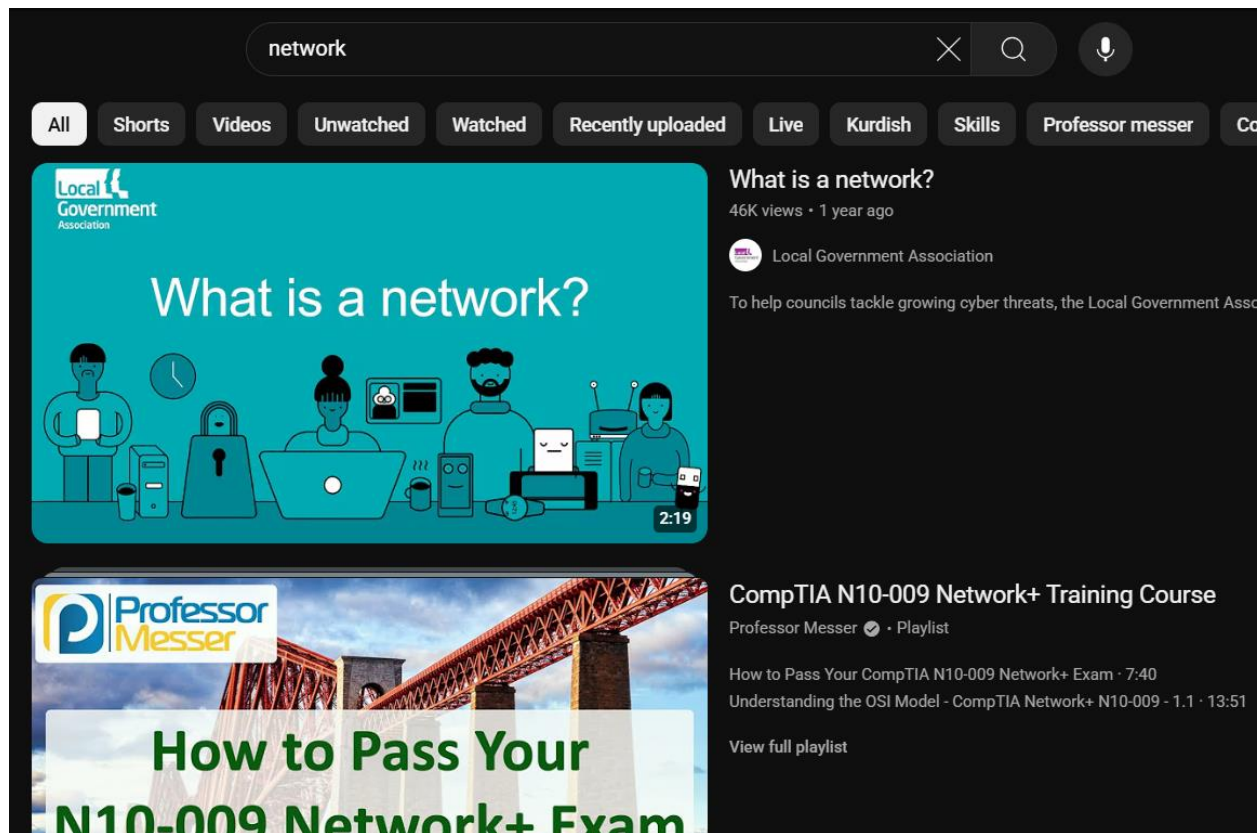


Figure 48: open YouTube after request doesn't exist video

4-MySite Arabic version:

127.0.0.1:9958/mySite_1220588_ar.html

طلب وسائط شبكات الحاسوب

اسم الملف:

مثال: Network-Edge

نوع الوسائط:

-- اختر نوع الوسائط --

طلب الوسائط

العودة للصفحة الرئيسية العربية | العودة للصفحة الرئيسية الإنجليزية | طلب الوسائط باللغة الإنجليزية

كيفية استخدام هذا النموذج:

أدخل اسم صورة أو فيديو شبكات الحاسوب الذي تبحث عنه واختر نوع الوسائط.
إذا لم يكن الملف متاحاً على خادمنا، سيتم توجيهك إلى نتائج بحث Google لطلبك.

- للصور: سيتم توجيهك إلى Google Images
- للفيديوهات: سيتم توجيهك إلى YouTube

Figure 49:mySite_1220588_ar.html

The result of run and search is the same with English version

Task 2 conclusion:

In this exercise, we were successful in installing and running a simple HTTP server using Python sockets to serve static HTML content along with associated media files. Upon startup of the server, it correctly listened for requests and served them effectively. Upon request for the main English HTML page, the server returned the correct content along with the appropriate status codes. Besides, all the images that were added into the page were fetched using individual requests, which were successfully processed and sent. This is proof that the server can process entire HTML web pages as well as supporting media resources in a reliable and well-managed manner. The terminal output confirmed correct functionality, safe file handling, and correct response formatting, setting a good platform for developing more interactive or dynamic web components in the next tasks.

Task 3 – TCP/UDP Hybrid Client-Server Game Using Socket Programming

In this task, we implemented a hybrid networked guessing game by using where a server uses TCP socket programming for control messages, reliable communication and UDP socket programming for real-time gameplay. The server waits for at least two players to join before starting the game. Once the required number of players is connected, the server sends a welcome message to all active clients. The game is divided into three rounds, each consisting of three questions. The server sends the same question to all players and waits for their answers within 60 seconds. The scoring system awards the player who answers correctly first with a full point, the second player with half a point, the third with a third of a point, and so on. The player with the highest score in each round wins that round. After all rounds are completed, the overall winner is the player who wins the most rounds. Players can join the game at any time, and any active player can leave by sending "exit" to the server.

- **Server code :**

```
1  import socket
2  import threading
3  import time
4  import random
5
6  # Server configuration
7  SERVER_HOST = '0.0.0.0'
8  TCP_PORT = 6000
9  UDP_PORT = 6001
10 GUESS_RANGE = (1, 100)
11 MIN_PLAYERS = 2
12 MAX_PLAYERS = 4
13 ROUND_DURATION = 60 # round time in seconds
14 # Get the server's IP address
15 hostname = socket.gethostname()
16 server_ip = socket.gethostbyname(hostname)
17 print(f"[SERVER] Server IP Address: {server_ip}")
18
19 players = {} # Dictionary to hold player name and TCP connection
20 player_addresses = {} # Map player names to (IP, UDP_port)
21 secret_number = None
22 game_active = False
23 start_time = None
24 round_timer = None
25 server_running = True
26 # Set up the TCP server socket
27 server_TCP_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
28 server_TCP_socket.bind((SERVER_HOST, TCP_PORT))
29 server_TCP_socket.listen()
30 print(f"[TCP] Listening on port {TCP_PORT}...")
31 # Set up the UDP server socket
32 server_UDP_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
33 server_UDP_socket.bind((SERVER_HOST, UDP_PORT))
34 print(f"[UDP] Listening for guesses on port {UDP_PORT}...")
35
```

Figure 50: set up UDP&TCP server.

In this part of the code, the first step is to import the essential libraries required for the implementation. The socket library is used to create network connections for communication

between the server and clients, specifically for TCP and UDP protocols. The threading library allows for concurrent execution, enabling the server to handle multiple tasks, such as receiving player connections and processing guesses, in parallel. The random library is utilized to randomly select a secret number for the game. Lastly, the time library is used to manage time-related operations, such as setting the duration of each game round.

After the necessary libraries are imported, we proceed to set up the server. The server is configured to listen on all available network interfaces by setting the `SERVER_HOST` to `'0.0.0.0'`. The server uses `TCP_PORT = 6000` for TCP communication (handling player connections and game status) and `UDP_PORT = 6001` for UDP communication (to receive guesses from players). The `GUESS_RANGE` is defined as `(1, 100)`, indicating the valid range for the secret number. The number of players allowed is controlled by `MIN_PLAYERS` and `MAX_PLAYERS`, with a minimum of 2 players and a maximum of 4. Each round of the game will last for `ROUND_DURATION = 60` seconds.

To ensure the server is accessible, we retrieve the server's IP address by calling `socket.gethostname()` to get the hostname of the machine, and `socket.gethostbyname()` to resolve it to an actual IP address. This IP address is printed so players can connect to the server.

Following this, we initialize several global variables that will be used to manage the game. These include:

- `players`, a dictionary that stores player names and their TCP connections.
- `player_addresses`, which maps player names to their respective IP addresses and UDP ports.
- `secret_number`, which will store the number the players need to guess.
- `game_active`, a flag to indicate whether the game is ongoing.
- `start_time`, to record when the round begins.
- `round_timer`, which will control the countdown for the round duration

```

36 # Function to handling TCP and UDP Connections
37 def handle_connections(protocol):
38     if protocol == 'TCP': # Handling TCP Connections
39         while server_running:
40             try:
41                 conn, addr = server_TCP_socket.accept()
42                 threading.Thread(target=handle_client_tcp, args=(conn, addr), daemon=True).start()
43             except Exception as e:
44                 if server_running: # Only log errors if server is supposed to be running
45                     print(f"[TCP ERROR] {str(e)}")
46     elif protocol == 'UDP': # Handling UDP Connections
47         while server_running:
48             try:
49                 data, addr = server_UDP_socket.recvfrom(1024)
50                 threading.Thread(target=handle_client_udp, args=(data, addr), daemon=True).start()
51             except Exception as e:
52                 if server_running: # Only log errors if server is supposed to be running
53                     print(f"[UDP ERROR] {str(e)}")
54
55 # Function to handle a TCP client connection
56 def handle_client_tcp(conn, addr):
57     global secret_number, game_active, start_time
58     name = None
59     try:
60         conn.send(b"Welcome! Please enter your name (e.g. Player_1=Last_First): ")
61         name_data = conn.recv(1024)
62         if not name_data:
63             conn.close()
64             return
65
66         name = name_data.decode().strip()
67
68         if name in players:
69             conn.send(b"Name already used. Try again.\n")
70             conn.close()
71             return
72
73         if len(players) >= MAX_PLAYERS:
74             conn.send(b"Maximum number of players reached.\n")
75             conn.close()
76             return
77
78         players[name] = conn
79         player_addresses[name] = addr[0] # Store just the IP, not the port since UDP port is different
80         print(f"[TCP] {name} joined from {addr[0]}")
81

```

Figure 51: Handling UDP&TCP connections

```

81
82     if len(players) < MIN_PLAYERS:
83         conn.send(b"Registered. Waiting for more players to join...\n")
84     else:
85         conn.send(b"Registered. Game is about to start or already started.\n")
86
87     if len(players) >= MIN_PLAYERS and not game_active:
88         threading.Thread(target=start_game, daemon=True).start()
89
90     if game_active:
91         conn.sendall(f"Game in progress! You can start guessing between {GUESS_RANGE[0]} and {GUESS_RANGE[1]} via UDP port {UDP_PORT}.\n".encode())
92
93     # Keep connection open to receive game updates
94     while server_running:
95         try:
96             data = conn.recv(1024)
97             if not data:
98                 raise ConnectionError("Connection closed")
99
100             # Handle any additional TCP commands here if needed
101
102         except Exception:
103             print(f"[DISCONNECT] {name} disconnected")
104             if name in players:
105                 del players[name]
106             if name in player_addresses:
107                 del player_addresses[name]
108             conn.close()
109             check_players_after_leave()
110             break
111
112     except Exception as e:
113         print(f"[ERROR] Connection lost with {addr}: {str(e)}")
114         if name and name in players:
115             del players[name]
116         if name and name in player_addresses:
117             del player_addresses[name]
118         conn.close()
119         check_players_after_leave()
120

```

Figure 52: Handling TCP clients

First, manage incoming connections based on the protocol (TCP or UDP). For TCP, it listens for client connections and creates a new thread to handle each client via the `handle_client_tcp` function, ensuring parallel processing. If the protocol is UDP, it listens for guesses and creates a thread to handle each client's guess via `handle_client_udp`. Errors are logged if the server is running. The `handle_client_tcp` function manages the interaction with an individual TCP client. It requests the player's name, checks for duplicates, and adds valid players to the game. If the player's name is valid and the required number of players is reached, the game starts. If the game is active, the client is notified and can start guessing via UDP. The function also handles disconnections and ensures proper cleanup of player data.

```

122 # Function to check the players' status
123 def check_players_after_leave():
124     global game_active, secret_number
125
126     if len(players) == 1 and game_active:
127         only_name = list(players.keys())[0]
128         only_conn = players[only_name]
129         try:
130             only_conn.sendall(b"You are the last player. Do you want to continue playing alone? (yes/no): ")
131             response = only_conn.recv(1024).decode().strip().lower()
132             if response == 'yes':
133                 only_conn.sendall(b"You will continue playing alone.\n")
134             else:
135                 only_conn.sendall(b"Game ended.\n")
136                 only_conn.close()
137                 del players[only_name]
138                 del player_addresses[only_name]
139                 end_game()
140         except Exception as e:
141             print(f"[ERROR] Could not communicate with {only_name}: {e}")
142
143     elif len(players) == 0 and game_active:
144         print("[INFO] No players remaining. Ending game.")
145         end_game()
146
147 # Function to handle UDP client guesses
148 def handle_client_udp(data, addr):
149     global secret_number, game_active, start_time
150
151     if not game_active or secret_number is None:
152         server_UDP_socket.sendto(b"No active game round. Wait for the next round to start.", addr)
153         return
154
155     # Check if the round time is up
156     time_elapsed = time.time() - start_time
157     if time_elapsed >= ROUND_DURATION:
158         end_round_by_timeout()
159         server_UDP_socket.sendto(b"Time's up! Round has ended.", addr)
160         return

```

Figure 53: Handling UDP client

Checks the status of the game if players leave. If only one player remains in the game and it is still active, the player is asked if they want to continue playing alone. If the player chooses not to continue, the game is ended, and that player is removed from the game. If no players remain, the game is automatically ended. This ensures the game doesn't continue without players and handles the edge case of a single player remaining. Next, handling the guesses sent by clients over UDP. If the game is not active or if there is no secret number, the player is informed that the game has not started yet. It checks if the round has timed out and ends it if necessary. The function then processes the player's guess by converting it to an integer. If the guess is valid, it compares it to the secret number and responds with whether the guess is "higher", "lower", or "correct". If the guess is correct, the winner is announced. This function ensures that players can interact with the game using UDP and that guesses are validated.

```

168
169     # Find which player made this guess by IP address
170     player_name = "Unknown"
171     for name, ip in player_addresses.items():
172         if ip == addr[0]:
173             player_name = name
174             break
175
176     print(f"[UDP] Guess {guess} from {player_name} ({addr[0]})")
177
178     if guess < secret_number:
179         server_UDP_socket.sendto(b"Higher", addr)
180     elif guess > secret_number:
181         server_UDP_socket.sendto(b"Lower", addr)
182     else:
183         server_UDP_socket.sendto(b"Correct!", addr)
184         announce_winner(player_name)
185
186     # Function to start a new game round
187     def start_game():
188         global secret_number, game_active, start_time, round_timer
189
190         # Cancel any existing timer
191         if round_timer:
192             round_timer.cancel()
193
194         secret_number = random.randint(*GUESS_RANGE)
195         game_active = True
196         start_time = time.time()
197         print(f"[GAME] Starting the game. Secret number is {secret_number}")
198
199         message = f"Game started! Guess the number between {GUESS_RANGE[0]} and {GUESS_RANGE[1]}. \n"
200         message += f"You have {ROUND_DURATION} seconds. Send your guesses via UDP to port {UDP_PORT}. \n"
201
202         for name, conn in players.items():
203             try:
204                 conn.sendall(message.encode())
205             except Exception as e:
206                 print(f"[TCP] Could not notify {name}: {e}")
207
208         # Start the round timer
209         round_timer = threading.Timer(ROUND_DURATION, end_round_by_timeout)
210         round_timer.daemon = True
211         round_timer.start()

```

Figure 54: function of game

Here , check if there is an existing timer and cancel it to avoid conflicts. Then, a random secret number is generated within the range defined by GUESS_RANGE, and the game is marked as active. The start time of the round is recorded using time.time() to track how long the round has been running. A message is prepared and sent to all registered players via their TCP connections, informing them that the game has started, the range of numbers they need to guess, and the duration of the round. If there is any issue sending the message, an error is logged.

```

212
213 # Function to announce the winner of the game round
214 def announce_winner(winner_name):
215     global game_active
216
217     if not game_active: # Prevent race conditions
218         return
219
220     print(f"[WINNER] {winner_name} guessed correctly!")
221     message = f"{winner_name} won the round by guessing the correct number! The number was {secret_number}.\n".encode()
222
223     for name, conn in players.items():
224         try:
225             conn.sendall(message)
226         except Exception as e:
227             print(f"[TCP] Could not notify {name}: {e}")
228
229     end_game()
230
231     # Wait a moment before starting a new game
232     threading.Timer(3, start_new_game).start()
233     #-----
234     # Function to end the round if the time limit is reached
235 def end_round_by_timeout():
236     global game_active
237
238     if not game_active: # Prevent race conditions
239         return
240
241     print("[TIMEOUT] Round ended: time limit reached. No winner.")
242     message = f"Time's up! No winner this round. The correct number was {secret_number}.\n".encode()
243
244     for name, conn in players.items():
245         try:
246             conn.sendall(message)
247         except Exception as e:
248             print(f"[TCP] Could not notify {name}: {e}")
249
250     end_game()
251
252     # Wait a moment before starting a new game
253     threading.Timer(3, start_new_game).start()
254     #-----

```

Figure 55:remaining of code

The `announce_winner` function is responsible for notifying all players when a winner is found in the game. Once a player guesses the correct number, the winner's name is displayed, and all players are informed via TCP that the winner has been determined. After the announcement, the current game is ended by calling `end_game()`. A brief pause is introduced before a new game starts using the `start_new_game` function. This ensures that the game doesn't immediately restart but gives players a moment to prepare for the next round. Next, handles the scenario where the round ends due to the time limit being reached. If no player has guessed the correct number before the time runs out, a message is sent to all players to notify them that the round has ended with no winner. The game is then ended by calling `end_game()`, and, as with the winner announcement, a brief pause is introduced before the next game begins.

```

254 #-----
255 # Function to end the game
256 def end_game():
257     global game_active, secret_number, round_timer
258     game_active = False
259     secret_number = None
260
261     # Cancel the round timer if it exists
262     if round_timer:
263         round_timer.cancel()
264         round_timer = None
265 #-----
266 # Function to start a new game
267
268 def start_new_game():
269     if len(players) >= MIN_PLAYERS:
270         start_game()
271     else:
272         print(f"[GAME] Not enough players ({len(players)}/{MIN_PLAYERS}) to start a new game.")
273         notify_waiting_for_players()
274 #-----
275 # Function to notify players that more players are needed to start the game
276 def notify_waiting_for_players():
277     message = f"Waiting for more players to join. Need at least {MIN_PLAYERS} players.\n".encode()
278
279     for name, conn in players.items():
280         try:
281             conn.sendall(message)
282         except Exception as e:
283             print(f"[TCP] Could not notify {name}: {e}")
284
285 # Start handling connections
286 tcp_thread = threading.Thread(target=handle_connections, args=('TCP',), daemon=True)
287 udp_thread = threading.Thread(target=handle_connections, args=('UDP',), daemon=True)
288 tcp_thread.start()
289 udp_thread.start()
290
291 print("[SERVER] Server is running...")
292
293 try:
294     while True:
295         time.sleep(1)
296 except KeyboardInterrupt:
297     print("\n[SERVER] Shutting down...")
298     server_running = False
299     server_TCP_socket.close()
300     server_UDP_socket.close()

```

Figure 56:remaining of code2

Stops the game by setting the `game_active` flag to `False` and resets the secret number to `None`. If there was a round timer running, it cancels it to ensure that no further actions are taken after the game has ended. Then, checks whether enough players are available to start a new game. If the number of players meets the minimum requirement, it starts a new game by calling `start_game()`. If there are not enough players, it notifies all connected players to wait until more players join. The `notify_waiting_for_players` function is responsible for sending a message to all players informing them that more players are needed to start the game. It ensures that players are kept in the loop while waiting for the game to start.

- Client code:

```

1  import socket
2  import threading
3  import time
4  import sys
5  import re
6  class GameClient:
7      def __init__(self): # Initializes the client
8          self.tcp_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
9          self.udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
10         self.running = True
11         self.game_active = False
12         self.server_ip = None
13         self.udp_port = None
14         self.cooldown = 10 # Reducing the cooldown between guesses from 10 to 2 seconds
15         #-----
16         # Connects to the server by obtaining the server IP and port
17         def connect_to_server(self):
18             print("\n=== Number Guessing Game Client ===")
19
20             # Get server info
21             while True:
22                 server_ip = input("Enter server IP: ").strip()
23                 if re.match(r"^\d{1,3}(\.\d{1,3}){3}$", server_ip):
24                     break
25                 print("Invalid IP address format. Try again.")
26
27             while True:
28                 try:
29                     tcp_port = int(input("Enter TCP port : ").strip() or "6000")
30                     udp_port = int(input("Enter UDP port : ").strip() or "6001")
31                     break
32                 except ValueError:
33                     print("Port must be a number. Try again.")
34
35             print(f"\nConnecting to server at {server_ip}...")
36             try:
37                 # Connect to server
38                 self.tcp_socket.connect((server_ip, tcp_port))
39                 self.server_ip = server_ip
40                 self.udp_port = udp_port
41                 print("Connected to server!")
42
43                 # Configure UDP socket to be non-blocking
44                 self.udp_socket.settimeout(5) # 5 second timeout for UDP responses
45
46                 return True
47
48             except ConnectionRefusedError:
49                 print(f"Error: Could not connect to server at {server_ip}:{tcp_port}")
50                 return False
51             except Exception as e:
52                 print(f"Error connecting to server: {str(e)}")
53                 return False
54         #-----

```

Figure 57: Initializes the game client

Initializes the game client by creating the necessary TCP and UDP sockets for communication with the server. It also sets up flags to control the game's flow, such as running, which keeps the main loop active, and game_active, which tracks whether the game is currently being played. The function also stores the server's IP address and the UDP port number for communication. Lastly, it defines a cooldown period of 10 seconds between guesses, ensuring players wait before making the next guess to prevent spamming. The connect_to_server function prompts the user for the server's IP address and port numbers for both TCP and UDP. It validates the IP address format using a regular expression and ensures the port numbers entered are valid integers. After obtaining the correct details, it attempts to establish a TCP connection with the server. If successful, it sets up the UDP socket with a timeout of 5 seconds, allowing the client to wait for responses from the server. The function returns True if the connection is successful, or False if there is an error.

```

53         return False
54     #-----
55     # Listens for TCP messages from the server and processes responses
56     def listen_tcp(self):
57         """Listen for TCP messages from server"""
58         while self.running:
59             try:
60                 msg = self.tcp_socket.recv(1024).decode()
61                 if not msg:
62                     print("Connection to server lost.")
63                     self.running = False
64                     break
65
66                 print(f"\n[SERVER] {msg}")
67
68                 if "Game started" in msg or "guess the number" in msg.lower():
69                     self.game_active = True
70                     print("[INFO] Game is active! You can now send guesses.")
71
72                 elif "won the round" in msg or "Time's up" in msg:
73                     print("[INFO] Round ended. Waiting for next round...")
74                     self.game_active = False
75
76                 elif "Do you want to continue playing alone" in msg:
77                     response = input("\nYou are the last player. Continue playing alone? (yes/no): ").strip().lower()
78                     self.tcp_socket.send(response.encode())
79
80                     if response != "yes":
81                         print("Exiting game...")
82                         self.running = False
83                         break
84
85             except ConnectionResetError:
86                 print("\nServer connection closed.")
87                 self.running = False
88                 break
89             except ConnectionAbortedError:
90                 print("\nServer connection aborted.")
91                 self.running = False
92                 break
93             except socket.error as se:
94                 print(f"\n[ERROR] Socket error: {se}")
95                 self.running = False
96                 break
97             except Exception as e:
98                 print(f"\n[ERROR] Unexpected error: {str(e)}")
99                 self.running = False
100                 break
101     #-----
102     # Sends guesses to the server via UDP

```

Figure 58: Listening from server.

Listening to messages from the server through the TCP connection. It processes messages related to the game, such as when the game starts, when the player wins, or when the round ends. If the game is active, it updates the `game_active` flag to `True` and allows the player to start guessing. If the game ends, it resets this flag. The function also handles special scenarios, such as when the last player is left in the game, prompting them whether they want to continue alone. It ensures that any disconnection or error in communication results in an appropriate shutdown of the client.

```

101 #-----
102 # Sends guesses to the server via UDP
103 def send_guesses(self):
104     """Send guesses to server via UDP"""
105     while self.running:
106         try:
107             if self.game_active:
108                 guess = input("\nEnter your guess (1-100) or 'exit' to quit: ").strip()
109
110                 if not self.running:
111                     break
112
113                 if guess.lower() == 'exit':
114                     print("Exiting game...")
115                     self.running = False
116                     break
117
118                 try:
119                     guess_num = int(guess)
120                     if 1 <= guess_num <= 100:
121                         print(f"Sending guess: {guess_num}")
122                         self.udp_socket.sendto(guess.encode(), (self.server_ip, self.udp_port))
123
124                         try:
125                             # Get UDP response
126                             response, _ = self.udp_socket.recvfrom(1024)
127                             response_text = response.decode()
128                             print(f"[SERVER] {response_text}")
129
130                             if "Correct" in response_text:
131                                 print("Congratulations! You guessed correctly!")
132
133                             if self.cooldown > 0:
134                                 print(f"Waiting for {self.cooldown} seconds before next guess...")
135                                 time.sleep(self.cooldown)
136
137                             except socket.timeout:
138                                 print("No response from server. Try again.")
139
140                             else:
141                                 print("Number must be between 1-100. Try again.")
142                             except ValueError:
143                                 print("Please enter a valid number.")
144                         else:
145                             # If game is not active, wait a bit before checking again
146                             time.sleep(0.5)
147                     except KeyboardInterrupt:
148                         print("\nInterrupted by user.")
149                         self.running = False
150                         break
151                     except Exception as e:
152                         print(f"Error sending guess: {str(e)}")
153                         time.sleep(1) # Wait before trying again
154 #-----

```

Figure 59: Listening from server.

Handles sending player guesses to the server via the UDP connection. The player is prompted to enter a guess between 1 and 100, and the guess is sent to the server. The function listens for the server's response, which may indicate whether the guess was "higher", "lower", or "correct". If the guess is correct, the player is congratulated. Additionally, the function implements a cooldown period, ensuring the player waits a set amount of time between guesses. It also handles invalid inputs, prompting the player to try again if the guess is out of range or not a valid number.

```

154 #-----
155 # loop to connect to the server,
156 def run(self):
157     """Main client loop"""
158     try:
159         # Connect to server
160         if not self.connect_to_server():
161             print("Failed to connect to server. Exiting.")
162             return
163
164         # Process welcome message and name registration
165         welcome = self.tcp_socket.recv(1024).decode()
166         print(f"[SERVER] {welcome}")
167
168         # Send player name
169         name = input("Enter your name: ").strip()
170         self.tcp_socket.send(name.encode())
171
172         # Get registration response
173         response = self.tcp_socket.recv(1024).decode()
174         print(f"[SERVER] {response}")
175
176         # Start TCP listener thread
177         tcp_thread = threading.Thread(target=self.listen_tcp)
178         tcp_thread.daemon = True
179         tcp_thread.start()
180
181         # Start guessing loop
182         self.send_guesses()
183
184     except KeyboardInterrupt:
185         print("\nExiting...")
186     except Exception as e:
187         print(f"\nError: {str(e)}")
188     finally:
189         self.cleanup()
190
191 def cleanup(self):
192     """Clean up resources"""
193     self.running = False
194     try:
195         self.tcp_socket.close()
196         self.udp_socket.close()
197     except:
198         pass
199     print("\nDisconnected from server. Goodbye!")
200
201 if __name__ == "__main__":
202     client = GameClient()
203     client.run()

```

Figure 60: Game Loop.

Starts by attempting to connect to the server using the `connect_to_server` function, and if successful, it processes the welcome message from the server. The function then sends the player's name to the server for registration. Once registration is complete, it starts the `listen_tcp` function in a separate thread to handle incoming server messages, while the player can begin sending guesses through the `send_guesses` function. The function also handles keyboard interrupts and ensures that resources are properly cleaned up if the client exits the game. The cleanup function is responsible for closing the client's TCP and UDP sockets when the game ends or when the client is interrupted. It sets the running flag to False to stop the game loop and ensures all resources are properly freed. The function attempts to close both the TCP and UDP sockets to terminate the connection to the server and then prints a message to notify the user that the client has been disconnected. This ensures a clean exit from the game, preventing any lingering connections or resources from being left open.

Test cases:

- **Exist:**The player_2 chooses to exit the game, leading to the disconnection and cleanup of the connection”After the player exited, the server removed him and didn’t send messages for him”.

but here we will ask the last player :!

*if he will continue alone. YES

```

PS C:\Users\SS> python C:\Users\SS\Desktop\clients.py

=== Number Guessing Game Client ===
Enter server IP: 192.168.1.16
Enter TCP port : 6000
Enter UDP port : 6001

Connecting to server at 192.168.1.16...
Connected to server!
[SERVER] Welcome! Please enter your name (e.g. Player_1=Last_First):
Enter your name: player_1
[SERVER] Registered. Game is about to start or already started.

[SERVER] Game in progress! You can start guessing between 1 and 100 via UDP port 6001.

[SERVER] Game started! Guess the number between 1 and 100.
You have 60 seconds. Send your guesses via UDP to port 6001.

[INFO] Game is active! You can now send guesses.

Enter your guess (1-100) or 'exit' to quit: 40
Sending guess: 40
[SERVER] Higher
Waiting for 10 seconds before next guess...

[SERVER] You are the last player. Do you want to continue playing alone? (yes/no):
You are the last player. Continue playing alone? (yes/no): yes

Enter your guess (1-100) or 'exit' to quit: 41
Sending guess: 41
[SERVER] Correct!
Congratulations! You guessed correctly!
Waiting for 10 seconds before next guess...

[SERVER] player_1 won the round by guessing the correct number! The number was 41.

[INFO] Round ended. Waiting for next round...

[SERVER] Waiting for more players to join. Need at least 2 players.
  
```

```

Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\SS> python C:\Users\SS\Desktop\server.py
[SERVER] Server IP Address: 192.168.1.16
[TCP] Listening on port 6000...
[UDP] Listening for guesses on port 6001...
[SERVER] Server is running...
[TCP] player_2 joined from 192.168.1.16
[TCP] player_1 joined from 192.168.1.16
[GAME] Starting the game. Secret number is 41
[UDP] Guess 40 from player_2 (192.168.1.16)
[DISCONNECT] player_2 disconnected
[UDP] Guess 41 from player_1 (192.168.1.16)
[WINNER] player_1 guessed correctly!
[GAME] Not enough players (1/2) to start a new game.
  
```

```

PS C:\Users\SS> python C:\Users\SS\Desktop\clients.py

=== Number Guessing Game Client ===
Enter server IP: 192.168.1.16
Enter TCP port : 6000
Enter UDP port : 6001

Connecting to server at 192.168.1.16...
Connected to server!
[SERVER] Welcome! Please enter your name (e.g. Player_1=Last_First):
Enter your name: player_2
[SERVER] Registered. Waiting for more players to join...

[SERVER] Game started! Guess the number between 1 and 100.
You have 60 seconds. Send your guesses via UDP to port 6001.

[INFO] Game is active! You can now send guesses.

Enter your guess (1-100) or 'exit' to quit: exit
Exiting game...

Server connection aborted.
  
```

Figure 61:test case exit

*If he doesn't want to play alone. NO

So the game will finish

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\SS> python C:\Users\SS\Desktop\clients.py

=== Number Guessing Game Client ===
Enter server IP: 192.168.1.16
Enter TCP port : 6000
Enter UDP port : 6001

Connecting to server at 192.168.1.16...
Connected to server!
[SERVER] Welcome! Please enter your name (e.g. Player_1=Last_First):
Enter your name: player_2
[SERVER] Registered. Game is about to start or already started.

[SERVER] Game in progress! You can start guessing between 1 and 100 via UDP
port 6001.

[SERVER] Game started! Guess the number between 1 and 100.
You have 60 seconds. Send your guesses via UDP to port 6001.

[INFO] Game is active! You can now send guesses.

Enter your guess (1-100) or 'exit' to quit: 40
Sending guess: 40
[SERVER] Higher
Waiting for 10 seconds before next guess...

[SERVER] You are the last player. Do you want to continue playing alone? (ye
s/no):

You are the last player. Continue playing alone? (yes/no): no
Exiting game...

Disconnected from server. Goodbye!

Windows PowerShell
Install the latest PowerShell for new features and improvements! https
://aka.ms/PSWindows

PS C:\Users\SS> python C:\Users\SS\Desktop\server.py
[SERVER] Server IP Address: 192.168.1.16
[TCP] Listening on port 6000...
[UDP] Listening for guesses on port 6001...
[SERVER] Server is running...
[TCP] player_1 joined from 192.168.1.16
[TCP] player_2 joined from 192.168.1.16
[GAME] Starting the game. Secret number is 46
[UDP] Guess 40 from player_1 (192.168.1.16)
[DISCONNECT] player_1 disconnected
[DISCONNECT] player_2 disconnected
[INFO] player_2 chose not to continue alone. Disconnecting them.
[INFO] No players remaining. Ending game.
[ERROR] Could not communicate with player_2: 'player_2'
[ERROR] Connection lost with ('192.168.1.16', 56181): 'player_2'

Windows PowerShell
Connecting to server at 192.168.1.16...
Connected to server!
[SERVER] Welcome! Please enter your name (e.g. Player_1=Last_First):
Enter your name: player_1
[SERVER] Registered. Waiting for more players to join...

[SERVER] Game started! Guess the number between 1 and 100.
You have 60 seconds. Send your guesses via UDP to port 6001.

[INFO] Game is active! You can now send guesses.

Enter your guess (1-100) or 'exit' to quit: exit
Exiting game...

Server connection aborted.

Disconnected from server. Goodbye!
PS C:\Users\SS>
```

Figure 62:test case Exist (B)

- Two players joined:Player_1 &Player_2 ,the winner was the player_1

```

Windows PowerShell
Enter server IP: 192.168.1.16
Enter TCP port : 6000
Enter UDP port : 6001
Connecting to server at 192.168.1.16...
Connected to server!
[SERVER] Welcome! Please enter your name (e.g. Player_1=Last_First):
Enter your name: player_2
[SERVER] Registered. Game is about to start or already started.

[SERVER] Game in progress! You can start guessing between 1 and 100 via UDP port 6001.
You have 60 seconds. Send your guesses via UDP to port 6001.

[INFO] Game is active! You can now send guesses.
Enter your guess (1-100) or 'exit' to quit: 90
Sending guess: 90
[SERVER] Higher
Waiting for 10 seconds before next guess...
Enter your guess (1-100) or 'exit' to quit: 95
Sending guess: 95
[SERVER] Higher
Waiting for 10 seconds before next guess...
Enter your guess (1-100) or 'exit' to quit: 99
Sending guess: 99
[SERVER] Lower
Waiting for 10 seconds before next guess...
[SERVER] player_1 won the round by guessing the correct number! The number was 96.
[INFO] Round ended. Waiting for next round...
[SERVER] Game started! Guess the number between 1 and 100.
You have 60 seconds. Send your guesses via UDP to port 6001.
[INFO] Game is active! You can now send guesses.
Enter your guess (1-100) or 'exit' to quit:
[SERVER] You are the last player. Do you want to continue playing alone? (yes/no):
no
Please enter a valid number.
You are the last player. Continue playing alone? (yes/no): no
Exiting game...

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows
PS C:\Users\SS> python C:\Users\SS\Desktop\server.py
[SERVER] Server IP Address: 192.168.1.16
[TCP] Listening on port 6000...
[UDP] Listening for guesses on port 6001...
[SERVER] Server is running...
[TCP] player_1 joined from 192.168.1.16
[GAME] Starting the game. Secret number is 96
[UDP] Guess 90 from player_1 (192.168.1.16)
[UDP] Guess 95 from player_1 (192.168.1.16)
[UDP] Guess 99 from player_1 (192.168.1.16)
[UDP] Guess 99 from player_1 (192.168.1.16)
[UDP] Guess 96 from player_1 (192.168.1.16)
[GAME] player_1 guessed correctly!
[INFO] player_1 won the round by guessing the correct number! The number was 96.
Waiting for 10 seconds before next guess...
[INFO] Round ended. Waiting for next round...
[SERVER] Game started! Guess the number between 1 and 100.
You have 60 seconds. Send your guesses via UDP to port 6001.
[INFO] Game is active! You can now send guesses.
Enter your guess (1-100) or 'exit' to quit: exit
  
```

Figure 63: test case: Two players join

- Duplicate names of players:** The client was disconnected because it tried to connect with the name player_1, which was already in use by another client. The server rejected the connection, and the client lost the connection.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\SS> python C:\Users\SS\Desktop\clients.py
=== Number Guessing Game Client ===
Enter server IP: 192.168.1.16
Enter TCP port : 6000
Enter UDP port : 6001
Connecting to server at 192.168.1.16...
Connected to server!
[SERVER] Welcome! Please enter your name (e.g. Player_1=Last_First):
Enter your name: player_1
[SERVER] Name already used. Try again.
Connection to server lost.
Disconnected from server. Goodbye!
PS C:\Users\SS>

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\SS> python C:\Users\SS\Desktop\server.py
[SERVER] Server IP Address: 192.168.1.16
[TCP] Listening on port 6000...
[UDP] Listening for guesses on port 6001...
[SERVER] Server is running...
[TCP] player_1 joined from 192.168.1.16

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\SS> python C:\Users\SS\Desktop\clients.py
=== Number Guessing Game Client ===
Enter server IP: 192.168.1.16
Enter TCP port : 6000
Enter UDP port : 6001
Connecting to server at 192.168.1.16...
Connected to server!
[SERVER] Welcome! Please enter your name (e.g. Player_1=Last_First):
Enter your name: player_1
[SERVER] Registered. Waiting for more players to join...
  
```

Figure 64: Duplicate names.

- Time's Up :** when the round time expired before any player could guess the correct number. The server sent the message "Time's up! No winner this round." because the round reached the time limit without a correct guess, and the game proceeded to the next round.

```
Windows PowerShell x Windows PowerShell x Windows PowerShell x + - □ x
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\SS> python C:\Users\SS\Desktop\clients.py

=== Number Guessing Game Client ===
Enter server IP: 192.168.1.16
Enter TCP port : 6000
Enter UDP port : 6001

Connecting to server at 192.168.1.16...
Connected to server!
[SERVER] Welcome! Please enter your name (e.g. Player_1-Last_First):
Enter your name: player_1
[SERVER] Registered. Game is about to start or already started.

[SERVER] Game in progress! You can start guessing between 1 and 100 via UDP port 6001.

[SERVER] Game started! Guess the number between 1 and 100.
You have 60 seconds. Send your guesses via UDP to port 6001.

[INFO] Game is active! You can now send guesses.

Enter your guess (1-100) or 'exit' to quit: 55
Sending guess: 55
[SERVER] Lower
Waiting for 10 seconds before next guess...

Enter your guess (1-100) or 'exit' to quit: 11
[SERVER] Time's up! No winner this round. The correct number was 48.

[INFO] Round ended. Waiting for next round...

[SERVER] Game started! Guess the number between 1 and 100.
You have 60 seconds. Send your guesses via UDP to port 6001.

[INFO] Game is active! You can now send guesses.

Windows PowerShell x Windows PowerShell x + - □ x
[UDP] Listening for guesses on port 6001...
[SERVER] Server is running...
[TCP] player_1 joined from 192.168.1.16
[TCP] player_2 joined from 192.168.1.16
[GAME] Starting the game. Secret number is 46
[UDP] Guess 40 from player_1 (192.168.1.16)
[DISCONNECT] player_1 disconnected
[DISCONNECT] player_2 disconnected
[INFO] player_2 chose not to continue alone. Disconnecting them.
[INFO] No players remaining. Ending game.
[ERROR] Could not communicate with player_2: 'player_2'
[ERROR] Connection lost with ('192.168.1.16', 56181): 'player_2'
[TCP] pp_fd joined from 192.168.1.16
[TCP] player_1 joined from 192.168.1.16
[GAME] Starting the game. Secret number is 48
[UDP] Guess 55 from pp_fd (192.168.1.16)
[UDP] Guess 11 from pp_fd (192.168.1.16)
[TIMEOUT] Round ended: time limit reached. No winner.
[GAME] Starting the game. Secret number is 60

Windows PowerShell x Windows PowerShell x + - □ x
Enter your guess (1-100) or 'exit' to quit: 11
Sending guess: 11
[SERVER] Higher
Waiting for 10 seconds before next guess...

Enter your guess (1-100) or 'exit' to quit:
[SERVER] Time's up! No winner this round. The correct number was 48.

[INFO] Round ended. Waiting for next round...

[SERVER] Game started! Guess the number between 1 and 100.
You have 60 seconds. Send your guesses via UDP to port 6001.

[INFO] Game is active! You can now send guesses.
Go to Settings to activate Windows.
```

Figure 65:time up

Alternative Solutions, Issues& Limitations

1. Alternative Approaches

- **task 1 – Network Commands and Wireshark**

Alternative Tool Usage:

Instead of using **only Wireshark**, we could have also used **tcpdump on Linux** as a lightweight command-line alternative for packet capture. Although it provides less visual detail than Wireshark, it's powerful for scripting and automation.

Automated Scripting:

Network commands like ping, traceroute, and nslookup could have been bundled into a script to run tests and gather results more efficiently, especially for repetitive diagnostics

- **Task 2 – Web Server Using Socket Programming**

For Task 2, several alternative approaches could improve the web server's efficiency and functionality. Instead of handling client requests sequentially, implementing a multithreaded or asynchronous server using select or asyncio would enable simultaneous handling of multiple clients, enhancing scalability. Additionally, replacing manual MIME type detection with Python's built-in mimetypes module would streamline content-type handling and increase compatibility with various file types. If not limited to raw sockets, using a lightweight web framework like Flask could further simplify development by supporting features like routing and POST requests, though this would shift the focus away from low-level socket programming.

- **Task 3 – TCP/UDP Multiplayer Guessing Game**

- **Pure TCP Design:**

Use TCP for both control messages and guessing to simplify protocol handling.

Pros: Easier code maintenance and reliable message delivery.

Cons: Higher latency during guessing due to TCP overhead and congestion control.

- **Multithreaded Client:**

Separate client tasks into multiple threads (e.g., TCP listening, UDP guessing, user input).

Pros: Better responsiveness and smoother gameplay.

Cons: Increased complexity and need for thread synchronization.

- **WebSockets or asyncio (if allowed):**

Use WebSockets for unified real-time communication or asyncio for scalable, non-blocking I/O.

Pros: Efficient handling of concurrent clients and real-time updates.

Cons: Requires external libraries and may go beyond project constraints.

2. Issues and Challenges

- **Task 2 – Arabic challenges**

While implementing and testing the server in Task Two, one of the main challenges we encountered was the server's inability to properly handle requests that included Arabic characters in the URL. When a request contained Arabic text such as accessing an Arabic HTML page or submitting a form with Arabic input the server failed to process it correctly and returned an HTTP 500 Internal Server Error. This occurred because the raw socket-based server was not fully equipped to decode or handle non-ASCII characters properly.

To resolve this issue, we ensured that all request data was decoded using UTF-8 and carefully validated query parameters and paths to avoid unexpected behavior caused by Unicode characters. We also tested the server with Arabic filenames and query inputs to confirm stability after applying the fix.

- **Task 3 – TCP/UDP Multiplayer Guessing Game**

- **TCP and UDP Coordination:** Synchronizing control messages (TCP) with real-time guesses (UDP) required careful timing, but it was handled successfully.
- **Thread Management:** Managing multiple threads for clients, timers, and game logic was challenging, but no major issues occurred.
- **UDP Behavior:** While UDP can lose or delay messages, the game logic handled it well, and gameplay was not affected.
- **Client Disconnections:** Handling players leaving mid-game added complexity, but the server managed these cases smoothly without errors.

3. Limitations

- **Task 2 Limitations:**

It doesn't support the handling of more than one client simultaneously (like., no multithreading or asynchronous I/O).

It can't handle complex form data or large file uploads.

The server has no support for full HTTP specification (like cookies, or POST request processing).

These limitations are to be expected for a low level, educational implementation, but mentioning them suggests areas where future releases might be refined.

- **Task 3 – TCP/UDP Multiplayer Guessing Game**

- **Localhost Limitation:** The game was tested and designed for clients and the server running on the same local machine. In real-world scenarios across different networks, additional configuration such as NAT traversal or port forwarding would be needed.
- **Limited Scalability:** The design was constrained to 2–4 players. Although the server uses threads to handle multiple clients, the implementation is not optimized for large-scale multiplayer games.
- **User Interface Simplicity:** The command-line interface, while functional, is minimal. A GUI or more interactive interface would improve usability and feedback clarity for users.
- **Single Round Design:** While voting was implemented to enable new rounds, the server architecture resets the game state after each round, which could be improved for smoother transitions and session management.

Teamwork

We worked collaboratively and distributed the tasks among ourselves in an organized and coordinated manner. Everyone took responsibility for their part of the work, and there was a strong

sense of harmony and teamwork. We were always ready to support each other to ensure the project was completed in the best possible way.

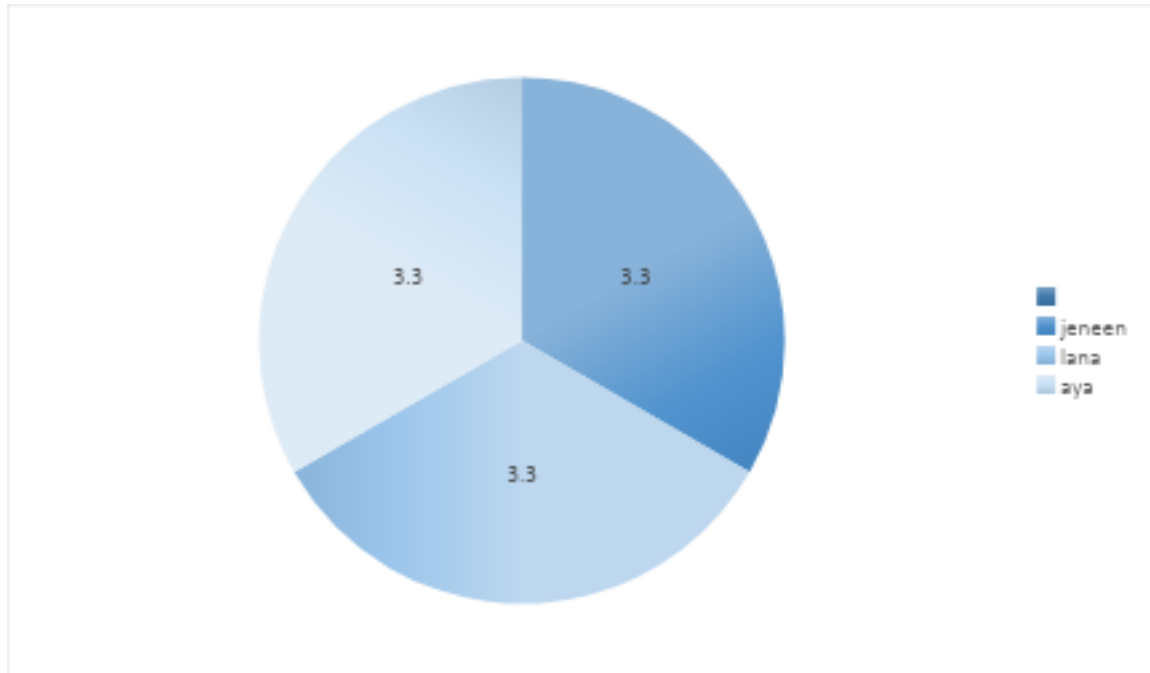


Figure 66: Teamwork

Conclusion

This project successfully enhanced our practical understanding of socket programming and computer networking through a series of well-structured tasks. By working with real network commands, analyzing traffic with Wireshark, building a custom web server, and developing a hybrid TCP/UDP guessing game, we were able to apply theoretical knowledge to real-world scenarios. Each task deepened our grasp of core networking principles such as communication protocols, data transmission, and client-server interaction. Overall, the project provided valuable hands-on experience and strengthened our problem-solving and teamwork skills in the context of computer networks.

Reference

- [1]: <https://www.scaler.com/topics/socket-programming-in-c/>
- [2]: <https://www.avast.com/c-tcp-vs-udp-difference>
- [3]: <https://www.atera.com/blog/network-commands/>
- [4]: <https://www.techtarget.com/whatis/definition/Web-server>
- [5]: <https://www.geeksforgeeks.org/web-servers-work/>